DWDM

Unit 1

What is Data Warehouse?

- Defined in many different ways, but not rigorously.
  - A decision support database that is maintained separately from the organization's operational database
  - Support information processing by providing a solid platform of consolidated, historical data for analysis.
- "A data warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision-making process."—W. H. Inmon

Data Warehouse—Subject-Oriented

- Organized around major subjects, such as customer, product, sales
- Focusing on the modeling and analysis of data for decision makers, not on daily operations or transaction processing
- Provide a simple and concise view around particular subject issues by excluding data that are not useful in the decision support process

Data Warehouse—Integrated

- Constructed by integrating multiple, heterogeneous data sources
  - relational databases, flat files, on-line transaction records
- Data cleaning and data integration techniques are applied.
  - Ensure consistency in naming conventions, encoding structures, attribute measures, etc. among different data sources
    - E.g., Hotel price: currency, tax, breakfast covered, etc.
  - When data is moved to the warehouse, it is converted.

Data Warehouse—Time Variant

- The time horizon for the data warehouse is significantly longer than that of operational systems
  - Operational database: current value data
  - Data warehouse data: provide information from a historical perspective (e.g., past 5-10 years)
- Every key structure in the data warehouse
  - Contains an element of time, explicitly or implicitly
  - But the key of operational data may or may not contain "time element"

Data Warehouse—Nonvolatile

- A physically separate store of data transformed from the operational environment
- Operational update of data does not occur in the data warehouse environment
  - Does not require transaction processing, recovery, and concurrency control mechanisms
  - Requires only two operations in data accessing:
    - *initial loading of data* and *access of data*

Functions of Data Warehousing

- **Data Extraction** - Involves gathering data from multiple heterogeneous sources.
- **Data Cleaning** - Involves finding and correcting the errors in data.
- **Data Transformation** - Involves converting the data from legacy format to warehouse format.
- **Data Loading** - Involves sorting, summarizing, consolidating, checking integrity, and building indices and partitions.
- **Refreshing** - Involves updating from data sources to warehouse.

DBMS vs. Data Warehouse

DBMS: (OLTP : Online Transaction Processing)

- **Query driven i.e.**
  - Build wrappers/mediators on top of heterogeneous databases
  - When a query is posed to a client site, a meta-dictionary is used to translate the query into queries appropriate for individual heterogeneous sites involved, and the results are integrated into a global answer set
- **OLTP i.e.**
  - Focuses on day to day operation of an organization such as purchasing, inventory, manufacturing, banking, payroll, registration, and accounting.

DBMS vs. Data Warehouse…

Data Warehouse: (OLAP: Online Analytical Processing)

- **Update driven i.e.**
  - Information from heterogeneous sources is integrated in advance and stored in warehouses for direct query and analysis
  - Such systems can organize and present data in various formats in order to accommodate the diverse needs of the different users.
- **OLAP i.e.**
  - serve users or knowledge workers in the role of data analysis and decision making.

DBMS vs. Data Warehouse…

	OLTP	OLAP
Users	clerk, IT professional	knowledge worker
Function	day to day operations	decision support
DB Design	application-oriented	subject-oriented
Data	current, up-to-date detailed, flat relational isolated	historical, summarized, multidimensional integrated, consolidated
Usage	repetitive	ad-hoc
Access	read/write index/hash on prim. key	lots of scans
Unit Of Work	short, simple transaction	complex query
# Records Accessed	tens	millions
#Users	thousands	hundreds
DB Size	100MB-GB	100GB-TB
Metric	transaction throughput	query throughput, response

Why Separate Data Warehouse?

- **High performance for both systems**
  - DBMS— tuned for OLTP: access methods, indexing, concurrency control, recovery
  - Warehouse—tuned for OLAP: complex OLAP queries, multidimensional view, consolidation
- **Different functions and different data:**
  - **Missing data:** Decision support requires historical data which operational DBs do not typically maintain
  - **Data consolidation:** DS requires consolidation (aggregation, summarization) of data from heterogeneous sources
  - **Data quality:** different sources typically use inconsistent data representations, codes and formats which have to be reconciled
- Note: There are more and more systems which perform OLAP analysis directly on relational databases

Schemas for Multidimensional Database

- Schema is a logical description of the entire database. It includes the name and description of records of all record types including all associated data-items and aggregates.
- A data warehouse uses
  - Star
  - Snowflake and
  - Fact Constellation schema.

Schemas for Multidimensional Database…

- **Star schema:** A fact table in the middle connected to a set of dimension tables

- **Snowflake schema:** A refinement of star schema where some dimensional hierarchy is normalized into a set of smaller dimension tables, forming a shape similar to snowflake

- **Fact constellations:** Multiple fact tables share dimension tables, viewed as a collection of stars, therefore called galaxy schema or fact constellation

Schemas for Multidimensional Database…

Star Schema:

- It is the data warehouse schema that contains two types of tables: *Fact Table and Dimension Tables.* Fact Table lies at the center point and dimension tables are connected with fact table such that star shape is formed.
- **Fact Tables:** A fact table typically has two types of columns: foreign keys to dimension tables and measures those that contain numeric facts. A fact table can contain data on detail or aggregated level.
- **Dimension Tables:** Dimension tables usually have a relatively small number of records compared to fact tables, but each record may have a very large number of attributes to describe the fact data.

Schemas for Multidimensional Database…

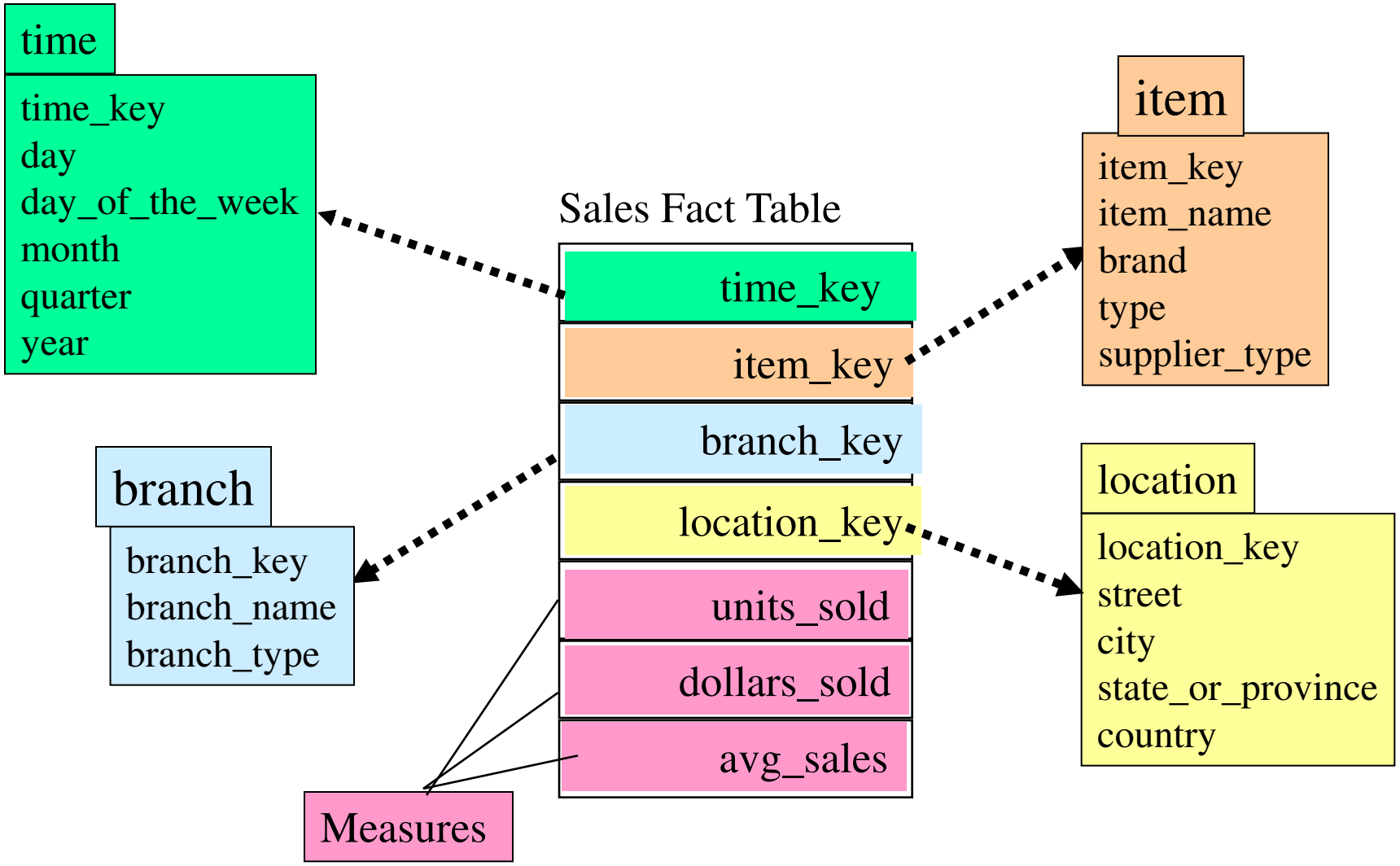


Figure: Example of Star Schema

Schemas for Multidimensional Database…

- Each dimension in the star schema has only one dimension table and each table holds a set of attributes. This constraint may cause data redundancy. The following diagram shows the sales data of a company with respect to the four dimensions, namely time, item, branch, and location.
- There is a fact table at the center. It contains the keys to each of four dimensions. The fact table also contains the attributes, namely dollars sold and units sold.
- Since star schema contains de-normalized dimension tables, it leads to simpler queries due to lesser number of join operations and it also leads to better system performance. On the other hand it is difficult to maintain integrity of data in star schema due to de-normalized tables. It is the widely used data warehouse schema and is also recommended by oracle

Schemas for Multidimensional Database…

Snowflake Schema

- The snowflake schema is a variant of the star schema model, where some dimension tables are *normalized*, thereby further splitting the data into additional tables.
- The resulting schema graph forms a shape similar to a snowflake.
- For example, the item dimension table in star schema is normalized and split into two dimension tables, namely item and supplier table.

Schemas for Multidimensional Database…

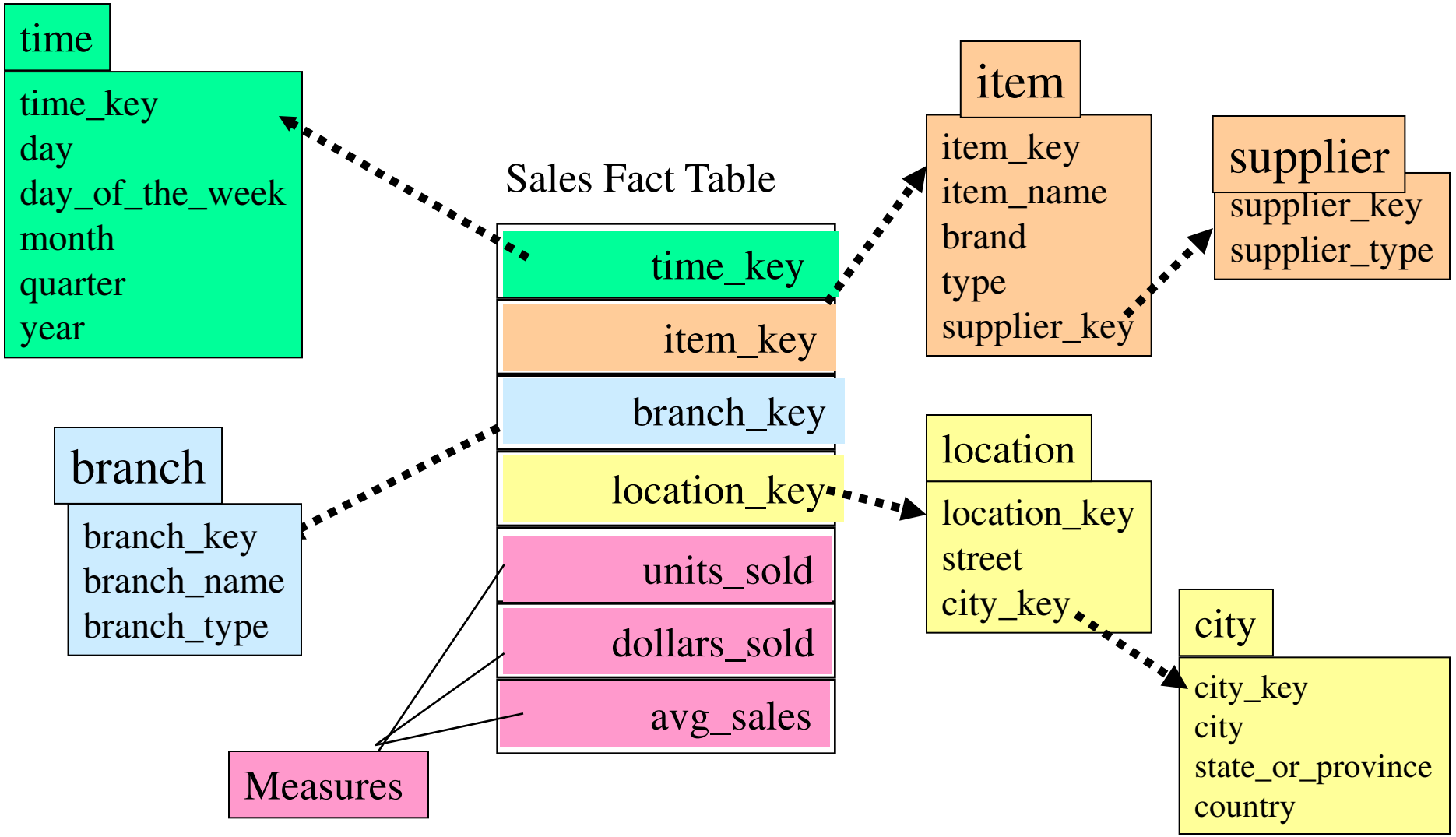


Figure: Example of Snowflake Schema

Schemas for Multidimensional Database…

- Due to normalization table is easy to maintain and saves storage space. However, this saving of space is negligible in comparison to the typical magnitude of the fact table.
- Furthermore, the snowflake structure can reduce the effectiveness of browsing, since more joins will be needed to execute a query.
- Consequently, the system performance may be adversely impacted.
- Hence, although the snowflake schema reduces redundancy, it is not as popular as the star schema in data warehouse design.

Schemas for Multidimensional Database…

Fact Constellation Schema

- This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation.
- A fact constellation schema allows dimension tables to be shared between fact tables.
- For example, following schema specifies two fact tables, sales and shipping. The sales table definition is identical to that of the star schema. The shipping table has five dimensions, or keys: item key, time key, shipper key, from location, and to location, and two measures: dollars cost and units shipped.

Schemas for Multidimensional Database…

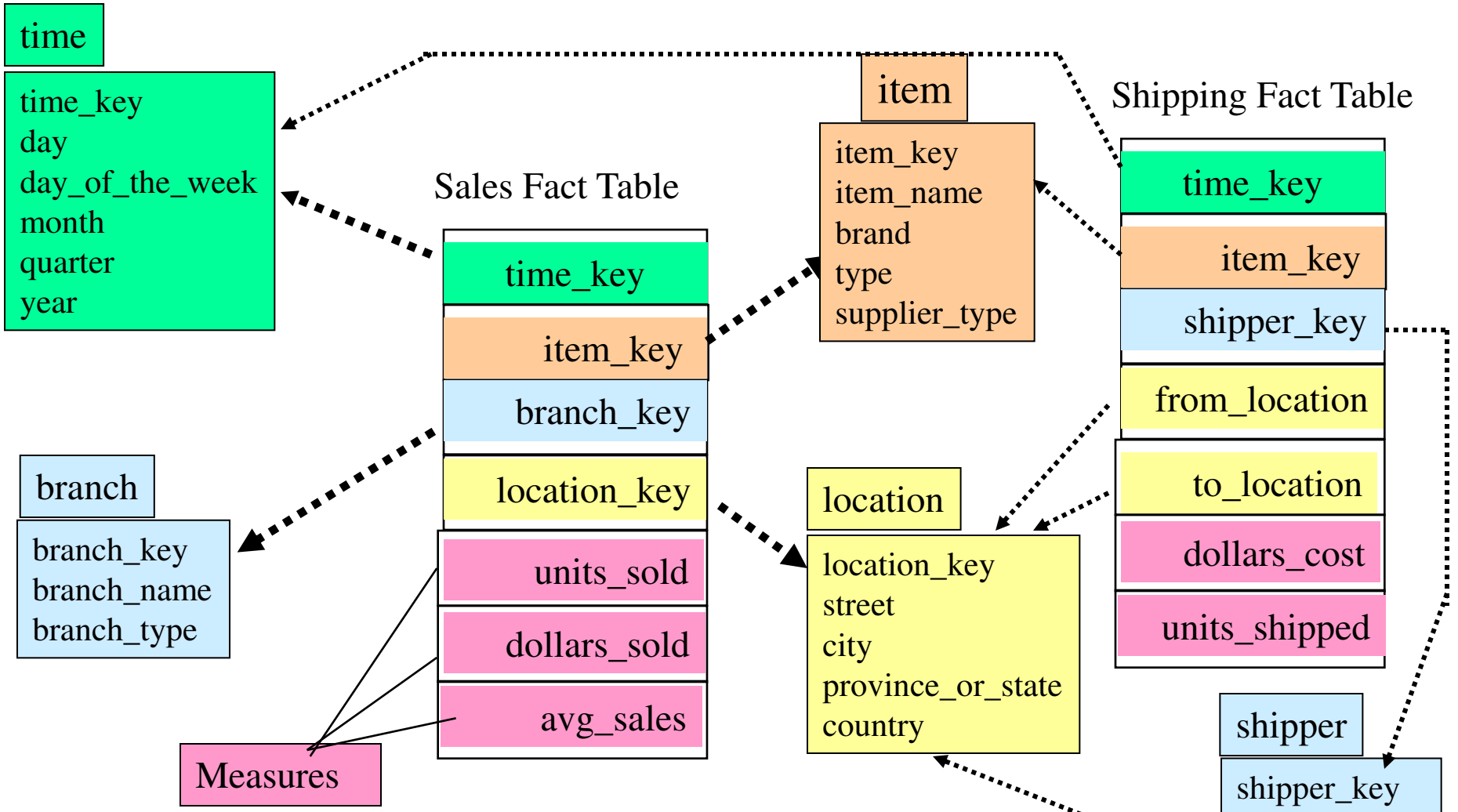


Figure: Example of Fac n

Cube Definition Syntax (BNF) in DMQL

- Cube Definition (Fact Table)

  define cube <cube_name> [<dimension_list>]:
  <measure_list>

- Dimension Definition (Dimension Table)

  define dimension <dimension_name**>** **as**
  (<attribute_or_subdimension_list>)

- Special Case (Shared Dimension Tables)
  - First time as “cube definition”
  - **define dimension** <dimension_name> **as**
    <dimension_name_first_time> **in cube**
    <cube_name_first_time>

Defining Star Schema in DMQL

define cube sales_star [time, item, branch, location]:

dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)

define dimension time **as** (time_key, day, day_of_week, month, quarter, year)

define dimension item **as** (item_key, item_name, brand, type, supplier_type)

define dimension branch **as** (branch_key, branch_name, branch_type)

define dimension location **as** (location_key, street, city, province_or_state, country)

Defining Snowflake Schema in DMQL

define cube sales_snowflake [time, item, branch, location]:

dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)

define dimension time **as** (time_key, day, day_of_week, month, quarter, year)

define dimension item **as** (item_key, item_name, brand, type, supplier(supplier_key, supplier_type))

define dimension branch **as** (branch_key, branch_name, branch_type)

define dimension location **as** (location_key, street, city(city_key, province_or_state, country))

Defining Fact Constellation in DMQL

define cube sales [time, item, branch, location]:

dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)

define dimension time **as** (time_key, day, day_of_week, month, quarter, year)

define dimension item **as** (item_key, item_name, brand, type, supplier_type)

define dimension branch **as** (branch_key, branch_name, branch_type)

define dimension location **as** (location_key, street, city, province_or_state, country)

Defining Fact Constellation in DMQL

define cube shipping [time, item, shipper, from_location, to_location]:

dollar_cost = sum(cost_in_dollars), unit_shipped = count(*)

define dimension time **as** time **in cube** sales

define dimension item time **as** item **in cube** sales

define dimension shipper **as** (shipper_key, shipper_name, location **as** location **in cube** sales, shipper_type)

define dimension from_location **as** location **in cube** sales

define dimension to_location **as** location **in cube** sales

Measures of Data Cube: Three Categories

- **Distributive:** if the result derived by applying the function to *n* aggregate values is the same as that derived by applying the function on all the data without partitioning
    - E.g., count(), sum(), min(), max()
- **Algebraic:** if it can be computed by an algebraic function with *M* arguments (where *M* is a bounded integer), each of which is obtained by applying a distributive aggregate function
    - E.g., avg(), min_N(), standard_deviation()
- **Holistic:** if there is no constant bound on the storage size needed to describe a subaggregate.
    - E.g., median(), mode(), rank()

Data Marts

- A data mart is a subject-oriented archive that stores data and uses the retrieved set of information to assist and support the requirements involved within a particular business function or department.

- Data marts exist within a single organizational data warehouse repository.

- Data marts improve end-user response time by allowing users to have access to the specific type of data they need to view most often.

Data Marts…

- A data mart is basically a condensed and more focused version of a data warehouse that reflects the regulations and process specifications of each business unit within an organization.

- Each data mart is dedicated to a specific business function or region.

- Different data marts can be used to obtain specific information for various enterprise departments, such as accounting, marketing, sales, etc.

Meta Data

- Metadata is simply defined as data about data.
- The data that is used to represent other data is known as metadata. For example, the index of a book serves as a metadata for the contents in the book.
- In other words, we can say that metadata is the summarized data that leads us to detailed data. In terms of data warehouse, we can define metadata as follows.
- Metadata is the road-map to a data warehouse.
- Metadata in a data warehouse defines the warehouse objects.
- Metadata acts as a directory. This directory helps the decision support system to locate the contents of a data warehouse.

Meta Data…

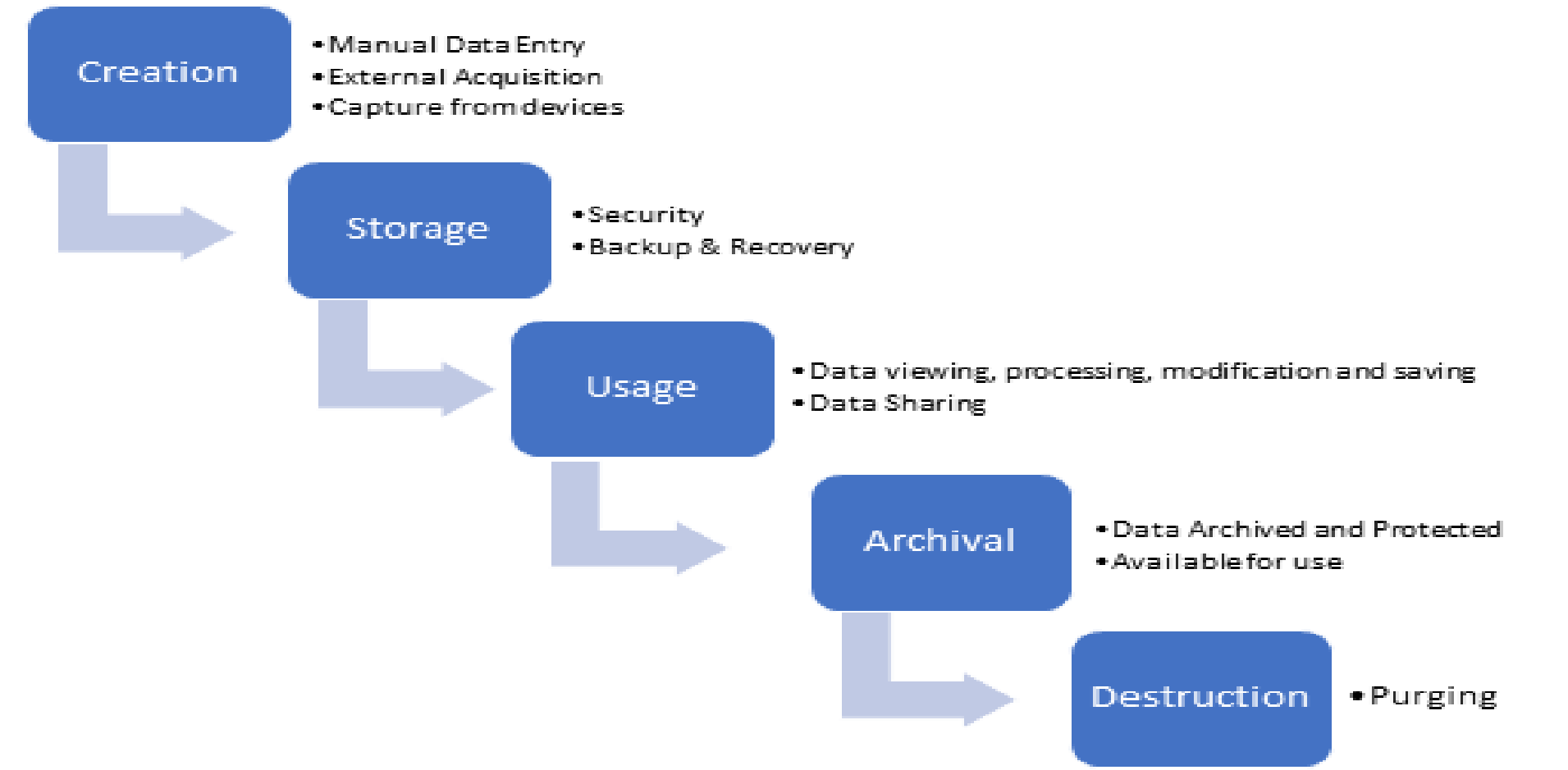
Metadata can be broadly categorized into three categories:

- **Business Metadata:** It has the data ownership information, business definition, and changing policies.
- **Technical Metadata -** It includes database system names, table and column names and sizes, data types and allowed values. Technical metadata also includes structural information such as primary and foreign key attributes and indices.
- **Operational Metadata** - It includes currency of data and data lineage. Currency of data means whether the data is active, archived, or purged. Lineage of data means the history of data migrated and transformation applied on it.

Lifecycle of Data

- The data life cycle is the sequence of stages that a particular unit of data goes through from its initial generation or capture to its eventual archival and/or deletion at the end of its useful life.
- Data Life Cycle Management is a process that helps organizations to manage the flow of data throughout its lifecycle – from initial creation through to destruction.
- While there are many interpretations as to the various phases of a typical data lifecycle, they can be summarized as follows:

Lifecycle of Data



Lifecycle of Data

1. Data Creation

- The first phase of the data lifecycle is the creation/capture of data. This data can be in many forms e.g. PDF, image, Word document, SQL database data. Data is typically created by an organization in one of 3 ways:
  - **Data Acquisition**: acquiring already existing data which has been produced outside the organization
  - **Data Entry**: manual entry of new data by personnel within the organization
  - **Data Capture**: capture of data generated by devices used in various processes in the organization

Lifecycle of Data

2. Storage

- Once data has been created within the organization, it needs to be stored and protected, with the appropriate level of security.
- A robust backup and recovery process should also be implemented to ensure retention of data during the lifecycle.

3.Usage

- During the usage phase of the data lifecycle, data is used to support activities in the organization.
- Data can be viewed, processed, modified and saved.
- An audit trail should be maintained for all critical data to ensure that all modifications to data are fully traceable.
- Data may also be made available to share with others outside the organization.

Lifecycle of Data

5. Archival

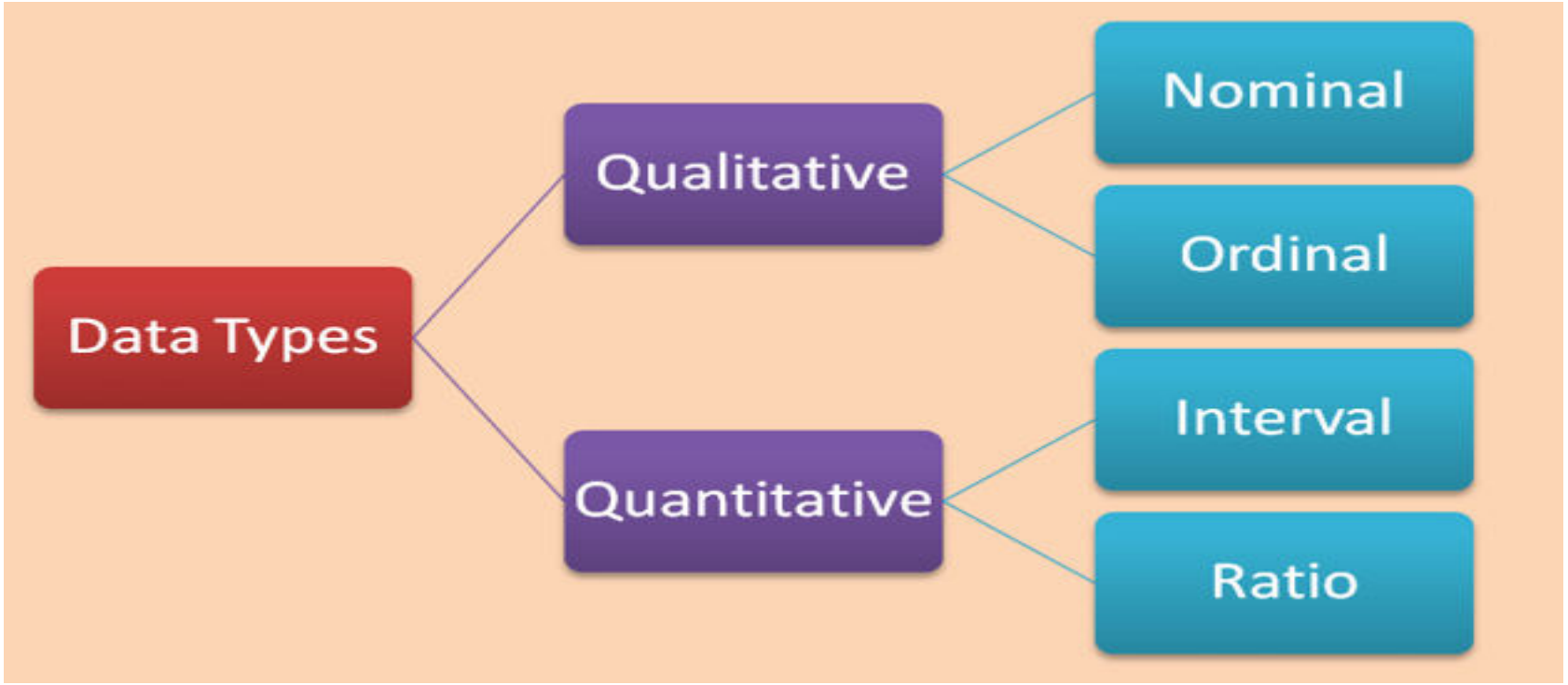
- Data Archival is the copying of data to an environment where it is stored in case it is needed again in an active production environment
- A data archive is simply a place where data is stored, but where no maintenance or general usage occurs.
- If necessary, the data can be restored to an environment where it can be used.

6. Destruction

- The volume of archived data inevitably grows, and while you may want to save all your data forever, that’s not feasible.
- It is typically done from an archive storage location.
- The challenge of this phase of the lifecycle is to ensure that the data has been properly destroyed.

Types of Data

- Basically, there are 2 classes of data in statistics that can be further sub-divided into 4 statistical data types.



Types of Data

- **Quantitative data** is information about quantities of things, things that we measure, and so we describe them in terms of numbers. As such, quantitative data are also called *Numerical data.*
- **Qualitative data** give us information about the qualities of things. They are observed phenomenon, not measured, and so we generally label them with names. Qualitative data are also known as *Categorical data.*

Types of Data

- **Nominal**
  - Examples: Nationality (British, American, Spanish,...), Genre/Style (Rock, Hip-Hop, Jazz, Classical,...), Favourite colour (red, green, blue,...)
- **Ordinal**
  - Examples: Opinion (agree, mostly agree, neutral, mostly disagree, disagree), Time of day (morning, noon, night)
- **Interval**
  - Examples: Temperature (°C or F, but not Kelvin), Dates (1066, 1492, 1776, etc.), Time interval on a 12 hour clock (6am, 6pm)
- **Ratio**
  - Differs from Interval data in that there is a *non-arbitrary zero-point* to the data
  - Examples: Age (from 0 years to 100+), Temperature (in Kelvin, but not °C or F), Distance (measured with a ruler or other such measuring device), Time interval (measured with a stop-watch or similar)

Properties of Attribute Values

- The type of an attribute depends on which of the following properties it possesses:
  - Distinctness: = ≠
  - Order: < >
  - Addition: + -
  - Multiplication: * /

  - Nominal attribute: distinctness
  - Ordinal attribute: distinctness & order
  - Interval attribute: distinctness, order & addition
  - Ratio attribute: all 4 properties

Attribute Type	Description	Examples	Operations
Nominal	The values of a nominal attribute are just different names, i.e., nominal attributes provide only enough information to distinguish one object from another. (=, ≠)	zip codes, employee ID numbers, eye color, sex: {*male, female*}	mode, entropy, contingency correlation, χ^2 test
Ordinal	The values of an ordinal attribute provide enough information to order objects. (<, >)	hardness of minerals, {*good, better, best*}, grades, street numbers	median, percentiles, rank correlation, run tests, sign tests
Interval	For interval attributes, the differences between values are meaningful, i.e., a unit of measurement exists. (+, -)	calendar dates, temperature in Celsius or Fahrenheit	mean, standard deviation, Pearson's correlation, *t* and *F* tests
Ratio	For ratio variables, both differences and ratios are meaningful. (*, /)	temperature in Kelvin, monetary quantities, counts, age, mass, length, electrical current	geometric mean, harmonic mean, percent variation

Attribute Level	Transformation	Comments
Nominal	Any permutation of values	If all employee ID numbers were reassigned, would it make any difference?
Ordinal	An order preserving change of values, i.e., *new_value = f(old_value)* where *f* is a monotonic function.	An attribute encompassing the notion of good, better best can be represented equally well by the values {1, 2, 3} or by { 0.5, 1, 10}.
Interval	*new_value =a * old_value + b* where a and b are constants	Thus, the Fahrenheit and Celsius temperature scales differ in terms of where their zero value is and the size of a unit (degree).
Ratio	*new_value = a * old_value*	Length can be measured in meters or feet.

Data Warehouse Architecture

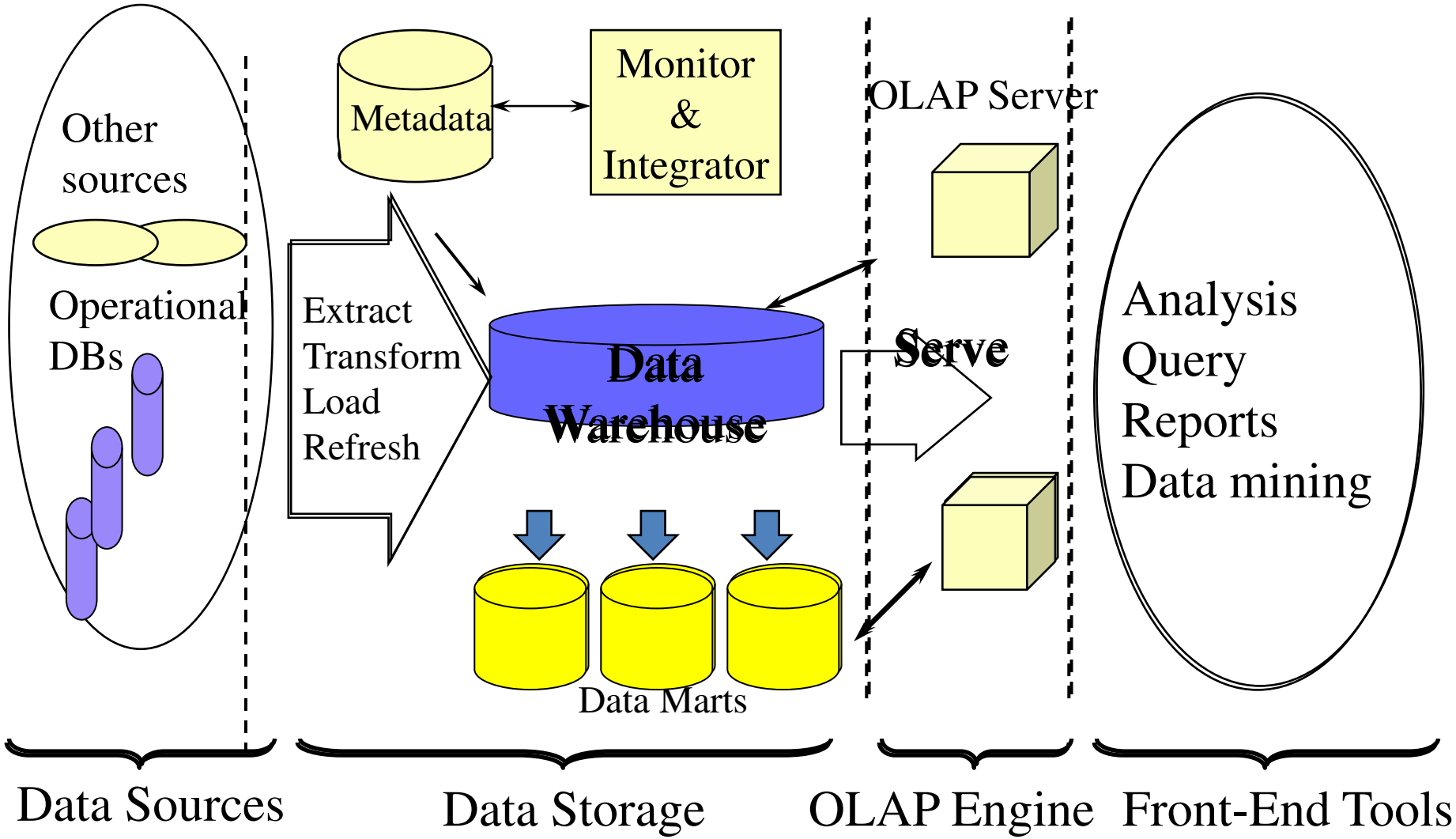


Figure: Data Warehouse: A Multi-Tiered Architecture

Data Warehouse Architecture…

Generally a data warehouses adopts three-tier architecture. Following are the three tiers of the data warehouse architecture.

Bottom Tier :

- The bottom tier of the architecture is the data warehouse database server.
- It is the relational database system or multidimensional model.
- Back end tools and utilities are used to feed data into the bottom tier.
- These back end tools and utilities perform the Extract, Clean, Load, and refresh functions.

Data Warehouse Architecture…

Generally a data warehouses adopts three-tier architecture. Following are the three tiers of the data warehouse architecture.

Bottom Tier :

- The bottom tier of the architecture is the data warehouse database server.
- It is the relational database system.
- Back end tools and utilities are used to feed data into the bottom tier.
- These back end tools and utilities perform the Extract, Clean, Load, and refresh functions.

Data Warehouse Architecture…

Middle Tier:

- In the middle tier, we have the OLAP Server that can be implemented in either of the following ways.
  - By Relational OLAP (ROLAP), which is an extended relational database management system. The ROLAP maps the operations on multidimensional data to standard relational operations.
  - By Multidimensional OLAP (MOLAP) model, which directly implements the multidimensional data and operations.
- Top-Tier :
  - This tier is the front-end client layer.
  - This layer holds the query tools and reporting tools, analysis tools and data mining tools.

Data Warehouse Models

From the perspective of data warehouse architecture, we have the following data warehouse models:

- Virtual Warehouse
- Data mart
- Enterprise Warehouse

Data Warehouse Models…

- Enterprise warehouse
  - collects all of the information about subjects spanning the entire organization
- Data Mart
  - a subset of corporate-wide data that is of value to a specific groups of users. Its scope is confined to specific, selected groups, such as marketing data mart
    - Independent vs. dependent (directly from warehouse) data mart
- Virtual warehouse
  - A set of views over operational databases
  - Only some of the possible summary views may be materialized

DWDM

Unit 2

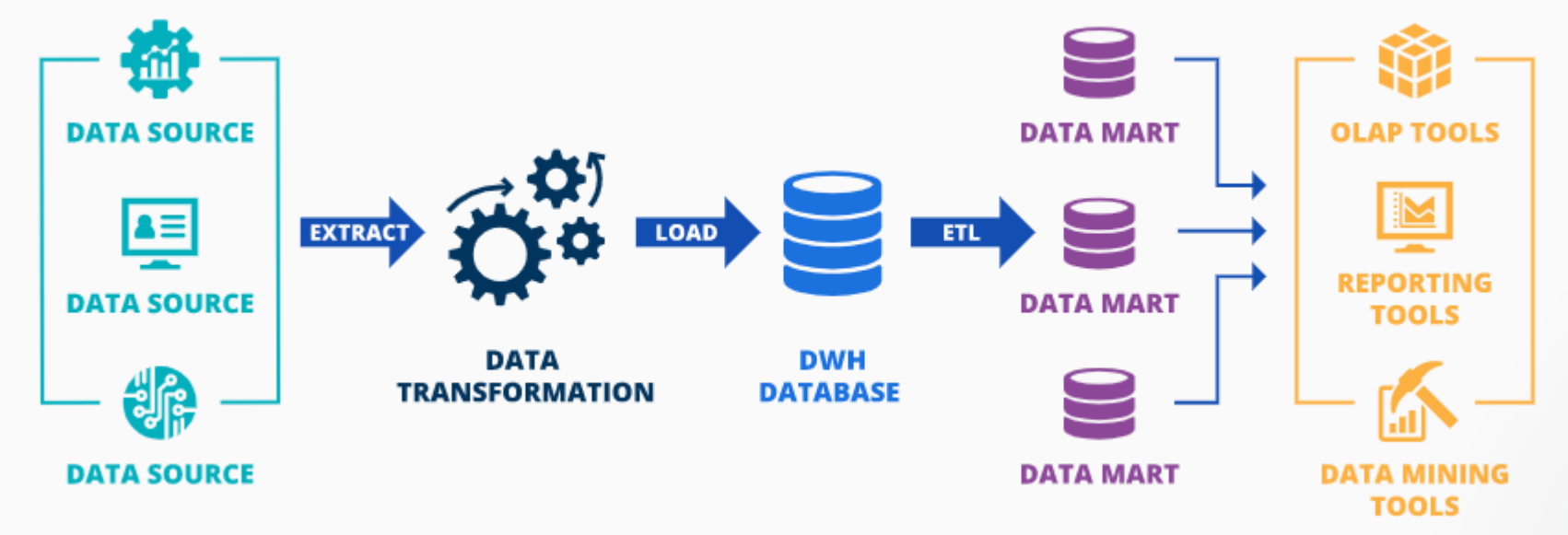
Layers within DW architecture

- Data Source Layer
  - Internal and external data sources
- Staging Layer
  - A temporary area where data transformations is carried out (may be absent if it is performed in data storage layer)
- Data Storage Layer
  - Consists of data warehouse database and data marts

Approaches to building DW

- Focuses on the data storage layer
- There are two approaches
  - Inmon's approach
  - Kimball's approach

Inmon’s approach to building DW



Pros and Cons of Inmon's approach

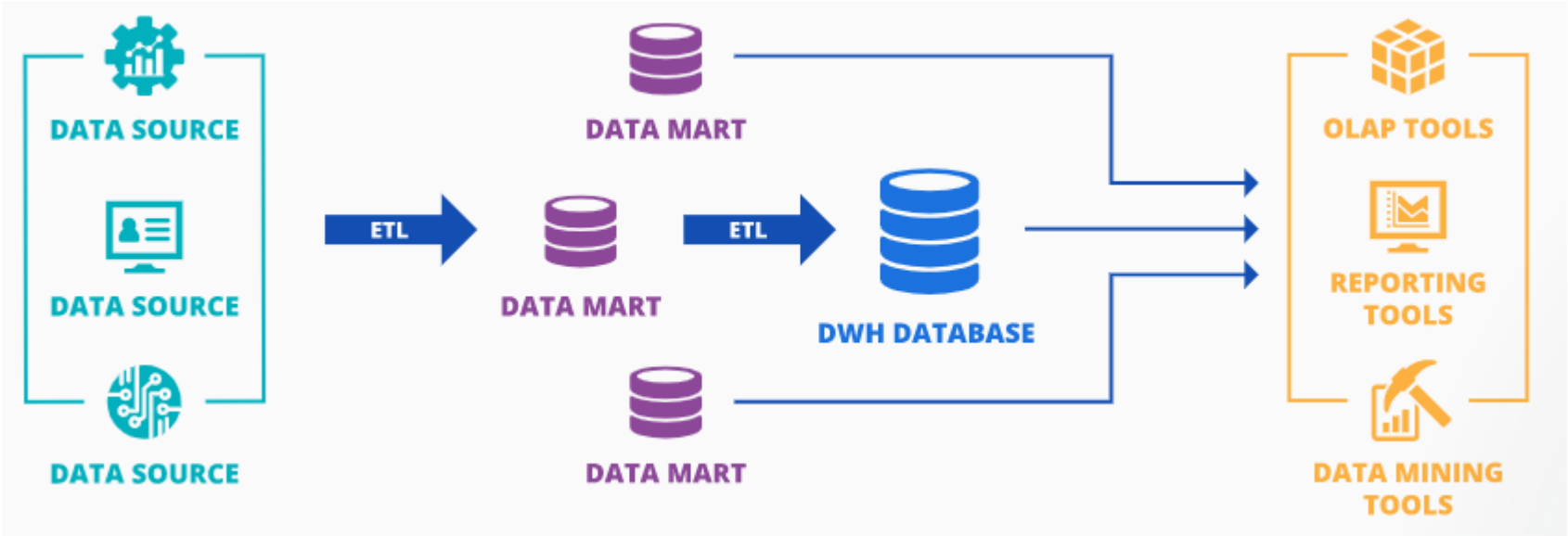
Pros

- Solid foundation for company-wide business Intelligence and data consistency across data marts.

Cons

- High initial costs and considerable time to build.

Kimball's approach to building DW



Pros and Cons of Kimball's approach

Pros

- Quick time-to-insight, facilitated analytics and reporting for individual business lines/teams.

Cons

- Possibility of redundant data and lack of data consistency in data marts as they are developed autonomously.

Populating DW

- It is an ongoing process
- Initial load + Periodic incremental loads
  - Full refresh may be required due to schema change
  - Facts table need more frequent updating than dimension table
- Eg. Db2, Apache Airflow, Apache Kafka, SQL, Python

Process of populating DW

- Choosing of appropriate extraction method
- Making the extracted data available for further processing

Extraction Methods

- Full Extraction
  - All the data is extracted completely from the source system since the last successful extraction
  - It does not require keeping track of changes in the source file
  - Mostly used for tables that will be used as dimensions table in a cube

Extraction Methods (contd…)

- Incremental Extraction
  - Only specific data that has changed from a specific time in history will be extracted
  - It requires keeping track of changes in the source file
  - Mostly used for tables that will be used as facts table in a cube

Extraction Methods (contd…)

- Full Extraction and Incremental Extraction can be online or offline

- Online
  - Direct access to source system

- Offline
  - No direct access to source system

Techniques for Tracking changes

- Change Data Capture (CDC)
  - It keeps log for every INSERTs, UPDATEs, and DELETEs on SQL Server tables
  - It keeps records of what changed, where and when
  - This log is then used as an input
  - Only available in Enterprise edition of SQL Server 2008 or later

Techniques for Tracking changes (contd…)

- Timestamps
  - It specifies the time and date that a given row was last modified
  - It makes easy to identify the latest data

- Partitioning
  - Implements range partitioning such that the source tables are partitioned along a date key

Techniques for Tracking changes (contd…)

- Database triggers
  - Triggers are added for INSERT, UPDATE AND DELETE

Techniques for Tracking changes (contd…)

- MERGE Statement
  - Extracting and entire table from source system and compare these tables with a previous extract from the source system to identify the changed data
  - All the fields of source need to be checked with the destination.
  - Or Hash functions like MD_5 can be used as well.

Unlocking the data assets to the end users

- Step 1
  - Eliminate data silos
- Step 2
  - Prioritize unified data access, security, and governance
- Step 3
  - Implement the right tools to analyze the data
- Step 4
  - Unlock new data with ML integration

Step 1

- liberate data from silos by moving it to a “data lake”—a central repository that allows to store all types of data in a single location at any scale.
- Eg, With AWS can improve organizational decision-making by moving any amount of data into AWS Simple Cloud Storage (S3).

Step 2

- control and visibility over the people who access data to satisfy both security and regulatory concerns.
- centrally control and manage security and governance policies, enabling the safe and compliant flow of data throughout the organization.

Step 3

- Cost, speed, and performance are top-of-mind concerns when it comes to converting data to actionable insights.
- Analytics tools need to be purpose-built to quickly generate insights from data, and analytics need to be optimized to deliver the best performance.

Step 4

- Add ML to data to improve operational efficiency, optimize existing processes, develop new products, and achieve revenue growth.
- ML integration also offers personalization that can improve customer engagement and conversion by creating a personalized experience for customers or end users.

Challenges for data unlocking

- Large amounts of data
- Siloed data storage
- Multiple formats
- Governance concerns
- Delayed time to insights
- Times and expense

DWDM

Unit 4

Multidimensional Data Model

From Tables and Spreadsheets to Data Cubes

- A data cube allows data to be modeled and viewed in multiple dimensions.
- It is defined by **dimensions** and **facts**.
- Dimensions are the entities with respect to which an organization wants to keep records.
- For example, an organization may create a *sales* data warehouse in order to keep records of the store's sales with respect to the dimensions *time*, *item*, *branch*, and *location*.
- Each dimension may have a table associated with it, called a *dimension table*. This table further describes the dimensions. For example, a dimension table for *item* may contain the attributes *item name, brand*, and *type*.

Multidimensional Data Model…

From Tables and Spreadsheets to Data Cubes

- A multidimensional data model is typically organized around a central theme, like *sales*. This theme is represented by a fact table.
- Facts are numerical measures. Themes are the quantities by which we want to analyze relationships between dimensions.
- Examples of facts for a sales data warehouse include *dollars_sold* (sales amount in dollars), *units_sold* (number of units sold), and *amount budgeted*.
- The fact table contains the names of the *facts*, or measures, as well as keys to each of the related dimension tables.

Multidimensional Data

	location = "Chicago"				location = "New York"				location = "Toronto"				location = "Vancouver"			
	item				item				item				item			
	home				home				home				home			
time	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Figure: Sales data for an organization according to the dimensions time, item, and location. The measure displayed is dollars_sold.

Multidimensional Data

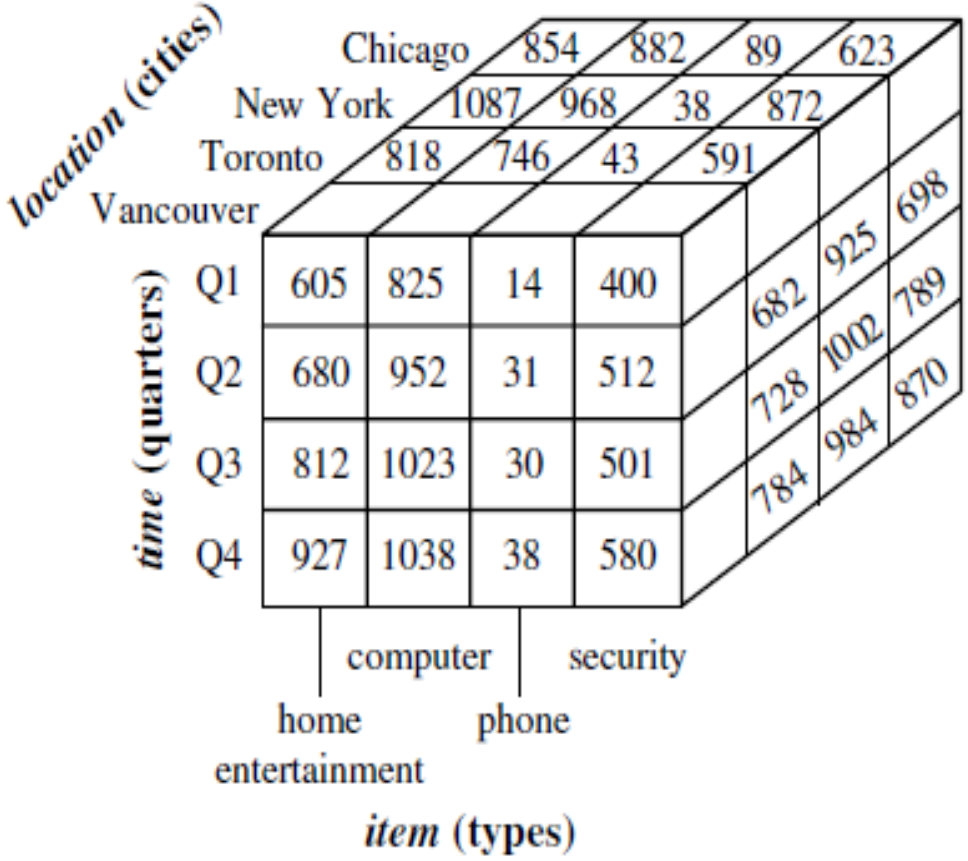


Figure A 3-D data cube representation of the data in above table, according to the dimensions time, item, and location. The measure displayed is dollars_sold (in thousands).

Multidimensional Data Model…

Suppose that we would now like to view our sales data with an additional fourth dimension, such as *supplier*. Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes, as shown in Figure below.

If we continue in this way, we may display any *n*-dimensional data as a series of
(*n*-1) dimensional cubes. The data cube is a metaphor for multidimensional data storage. The actual physical storage of such data may differ from its logical representation. The important thing to remember is that data cubes are
n-dimensional and do not confine data to 3-D.

Multidimensional Data

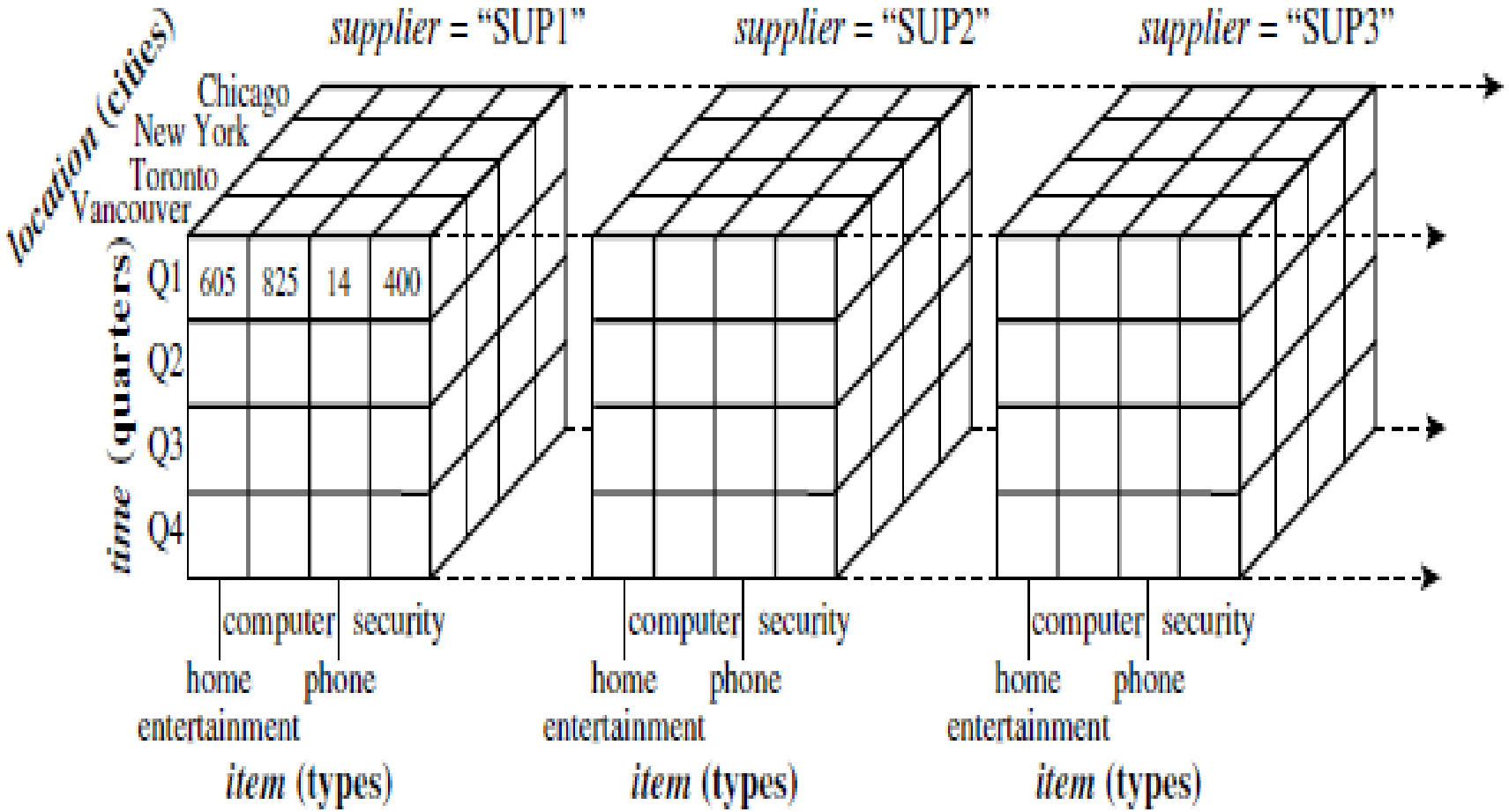


Figure 4-D data cube representation of sales data, according to the dimensions time, item, location, and supplier. The measure displayed is

Multidimensional Data Model…

- A data warehouse is based on a multidimensional data model which views data in the form of a data cube
- A data cube, such as sales, allows data to be modeled and viewed in multiple dimensions
  - Dimension tables, such as item (item_name, brand, type), or time(day, week, month, quarter, year)
  - Fact table contains measures (such as dollars_sold) and keys to each of the related dimension tables
- In data warehousing literature, an n-D base cube is called a base cuboid. The top most 0-D cuboid, which holds the highest-level of summarization, is called the apex cuboid. The lattice of cuboids forms a data cube.

Multidimensional Data Model…

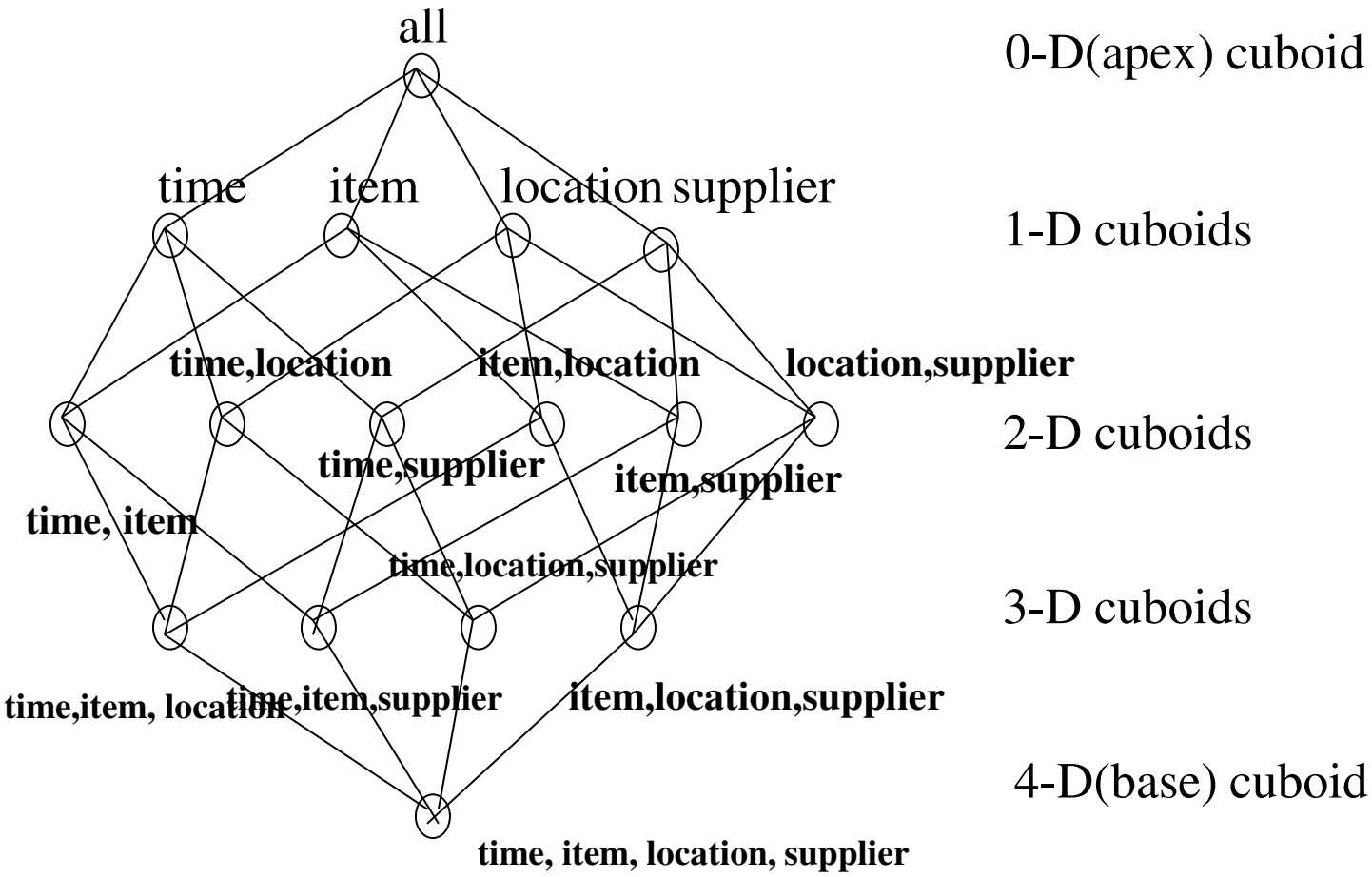
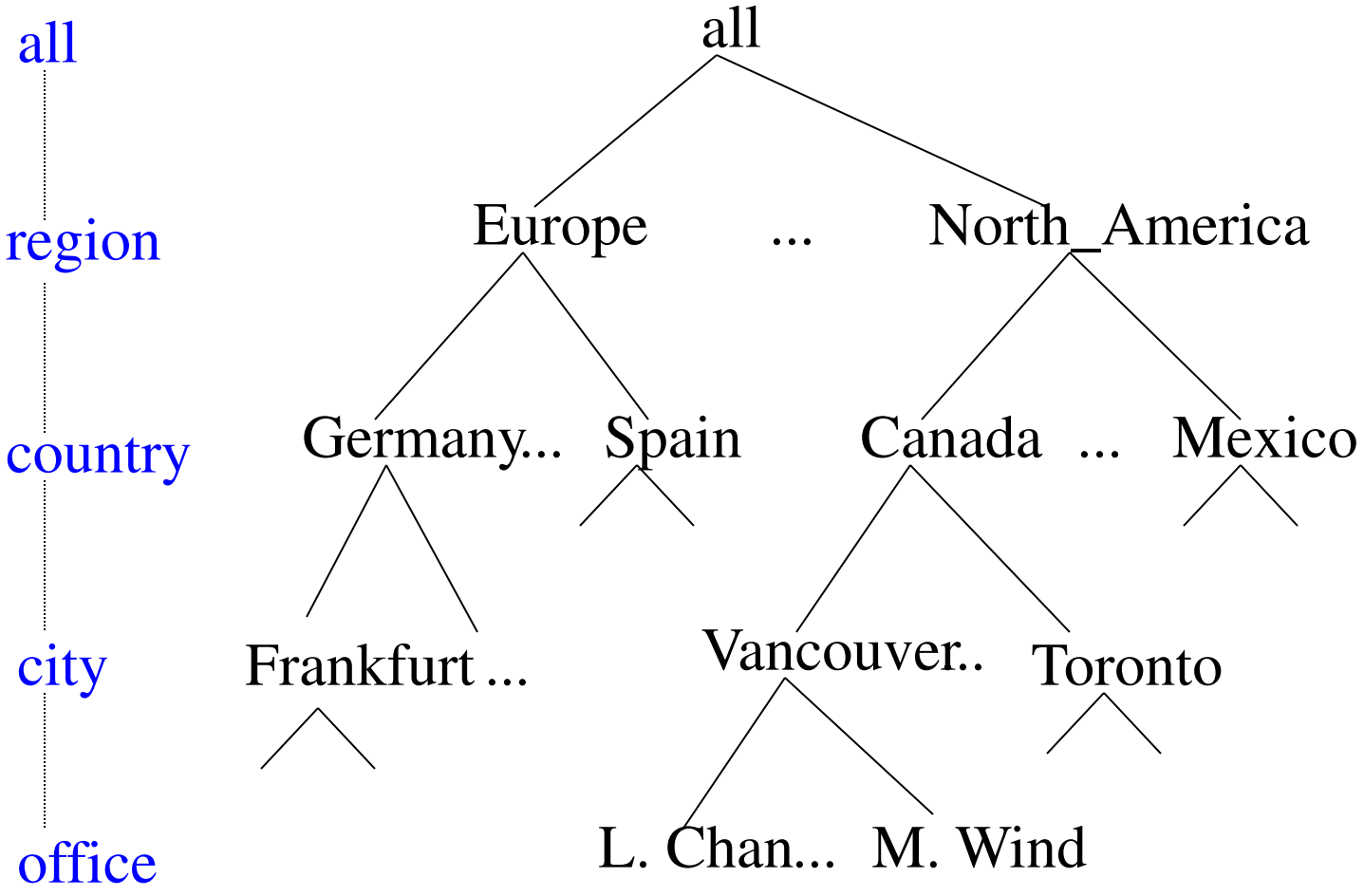


Figure: Cube; A Lattice of Cuboids

A Concept Hierarchy: Dimension (location)



Online Analytical Processing (OLAP)

Online Analytical Processing Server (OLAP) is based on the multidimensional data model.

It allows managers, and analysts to get an insight of the information through fast, consistent, and interactive access to information.

OLAP Operations

Since OLAP servers are based on multidimensional view of data, we will discuss OLAP operations in multidimensional data.

Here is the list of OLAP operations:

- Roll-up
- Drill-down
- Slice and dice
- Pivot (rotate)

OLAP Operations

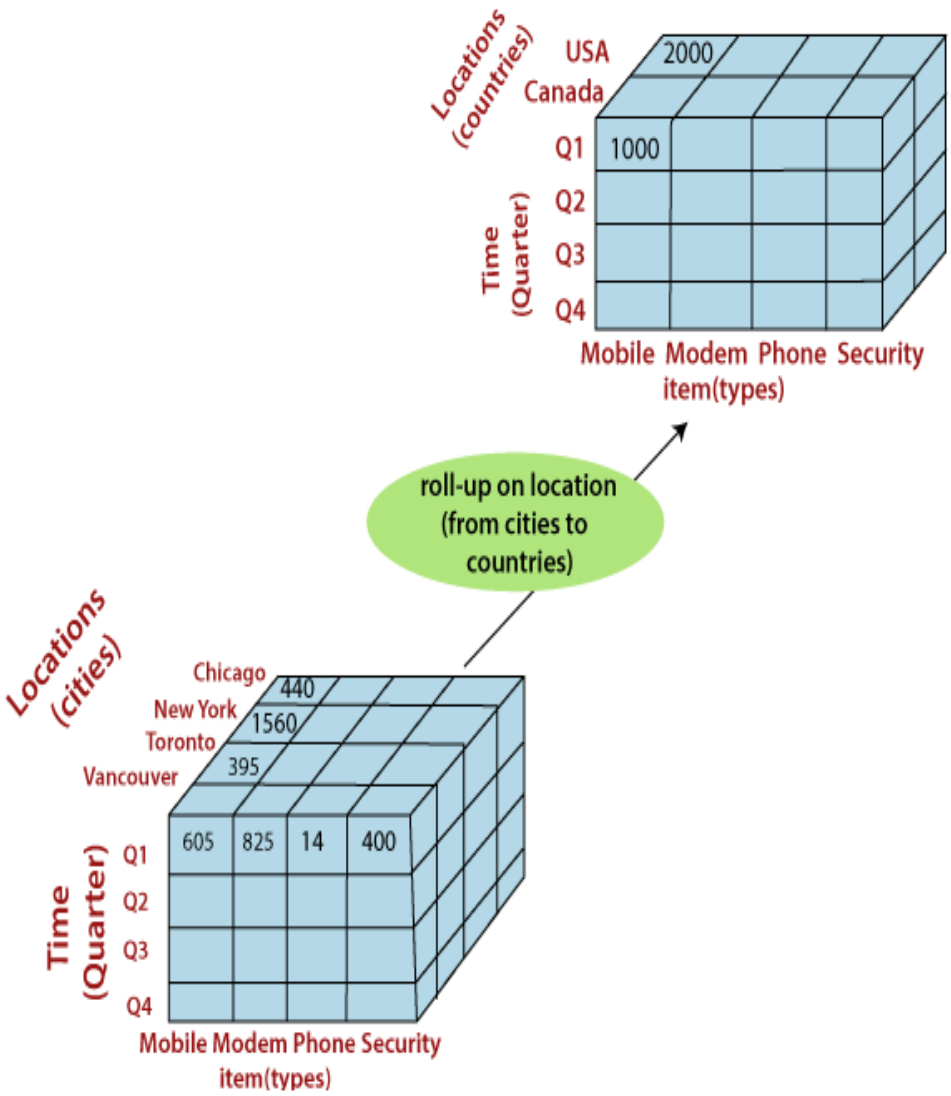


Figure: Roll up from City to Countries

OLAP Operations

Roll-up

- Roll-up performs aggregation on a data cube in any of the following ways:
- By climbing up a concept hierarchy for a dimension or by dimension reduction. The following diagram illustrates how roll-up works.
- Roll-up is performed by climbing up a concept hierarchy for the dimension location.
- Initially the concept hierarchy was "street < city < province < country". On rolling up, the data is aggregated by ascending the location hierarchy from the level of city to the level of country. The data is grouped into cities rather than countries.
- When roll-up is performed, one or more dimensions from the data cube are removed.

OLAP Operations

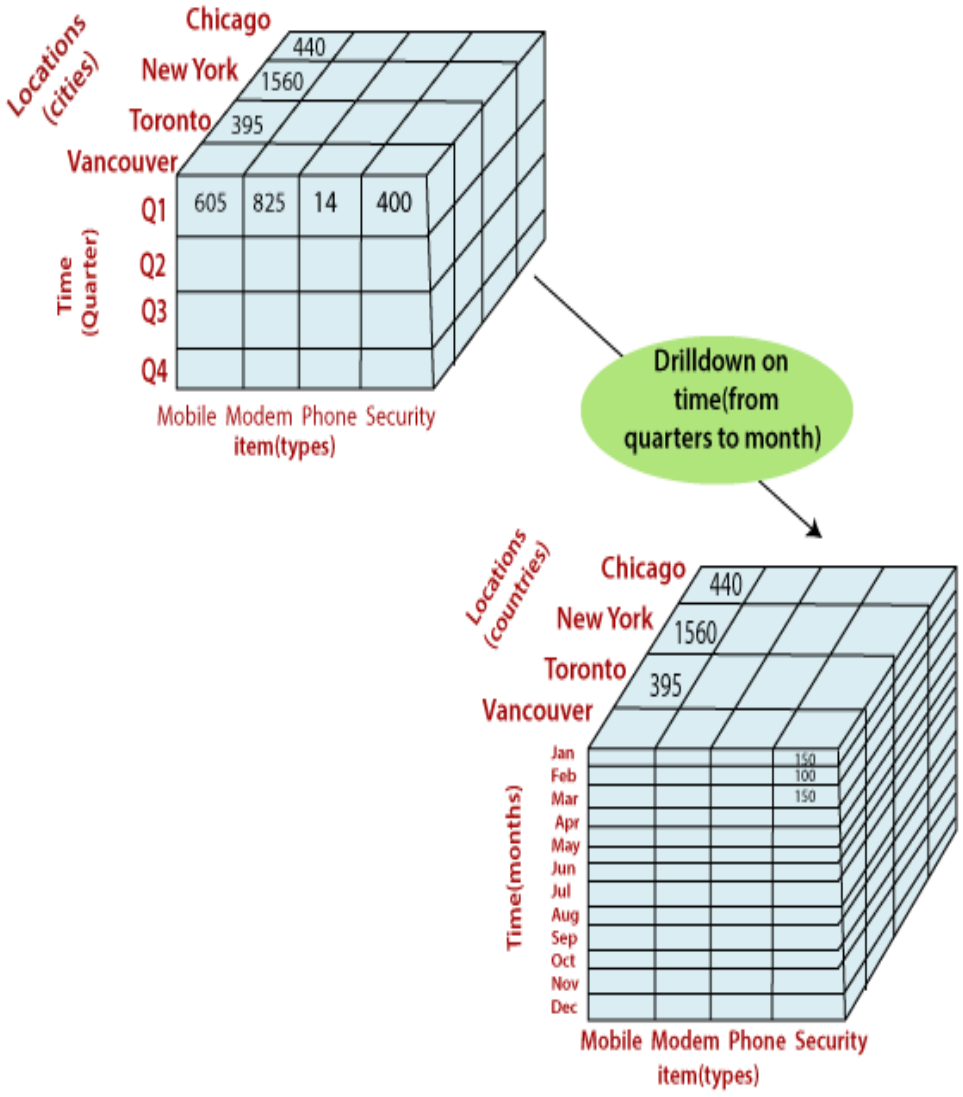


Figure: Drill down from Quarterly to months

OLAP Operations

Drill-down

- Drill-down is the reverse operation of roll-up.
- It is performed by either of the following ways:
- By stepping down a concept hierarchy for a dimension or by introducing a new dimension
- Drill-down is performed by stepping down a concept hierarchy for the dimension time. Initially the concept hierarchy was "day < month < quarter < year.“
- On drilling down, the time dimension is descended from the level of quarter to the level of month.
- When drill-down is performed, one or more dimensions from the data cube are added.
- It navigates the data from less detailed data to highly detailed data.

OLAP Operatior

Slice

- The slice operation selects one particular dimension from a given cube and provides a new sub-cube.
- Consider the following diagram that shows how slice works.

- Here Slice is performed for the dimension "time" using the criterion time = "Q1". It will form a new sub-cube by selecting one

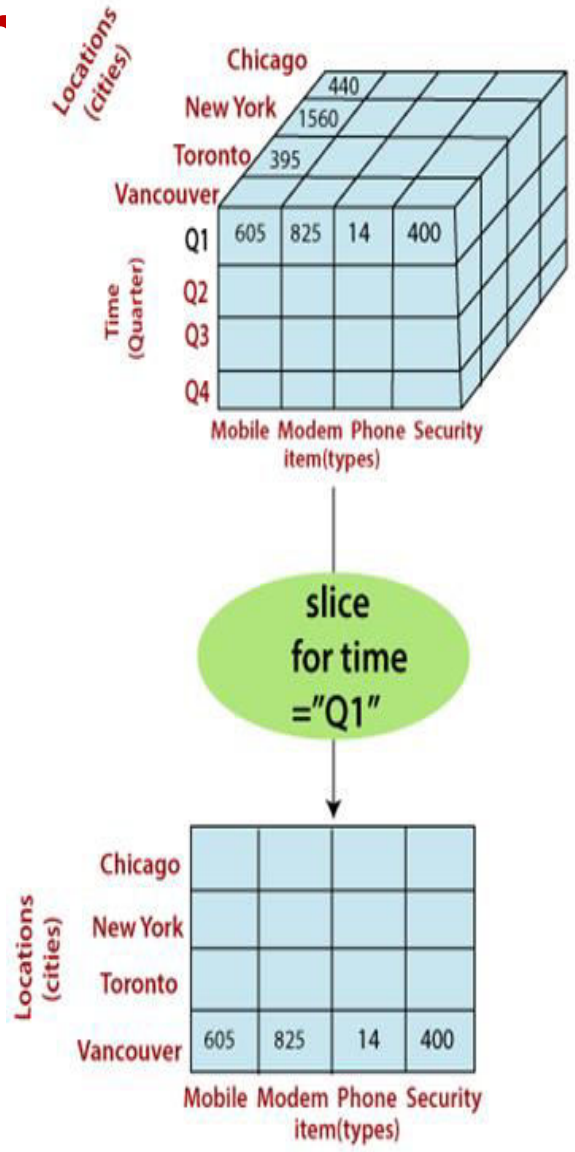


Figure: Performing slice op

OLAP Operatio

Dice

- Dice selects two or more dimensions from a given cube and provides a new sub-cube.
- Consider the following diagram that shows the dice operation.
- The dice operation on the cube based on the following selection criteria involves three dimensions.
- (location = "Toronto" or "Vancouver")
- (time = "Q1" or "Q2")
- (item =" Mobile" or "Modem")

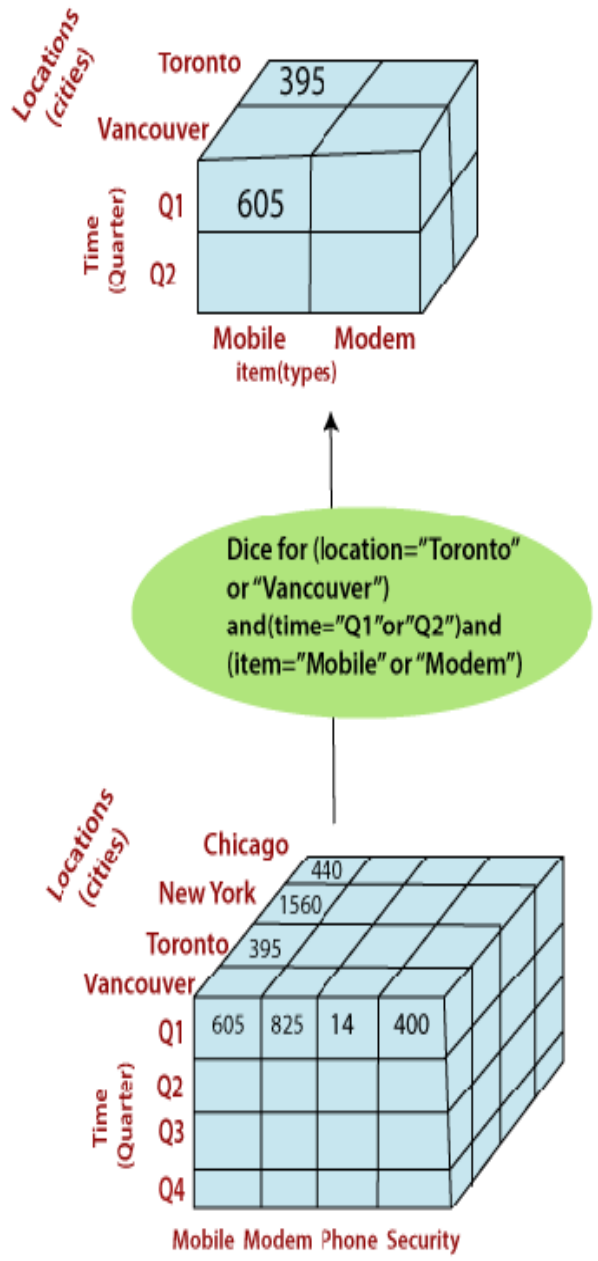


Figure: Performing dice op

OLAP Operations

Pivot

- The pivot operation is also called a rotation.
- Pivot is a visualization operations which rotates the data axes in view to provide an alternative presentation of the data.
- It may contain swapping the rows and columns or moving one of the row-dimensions into the column dimensions.

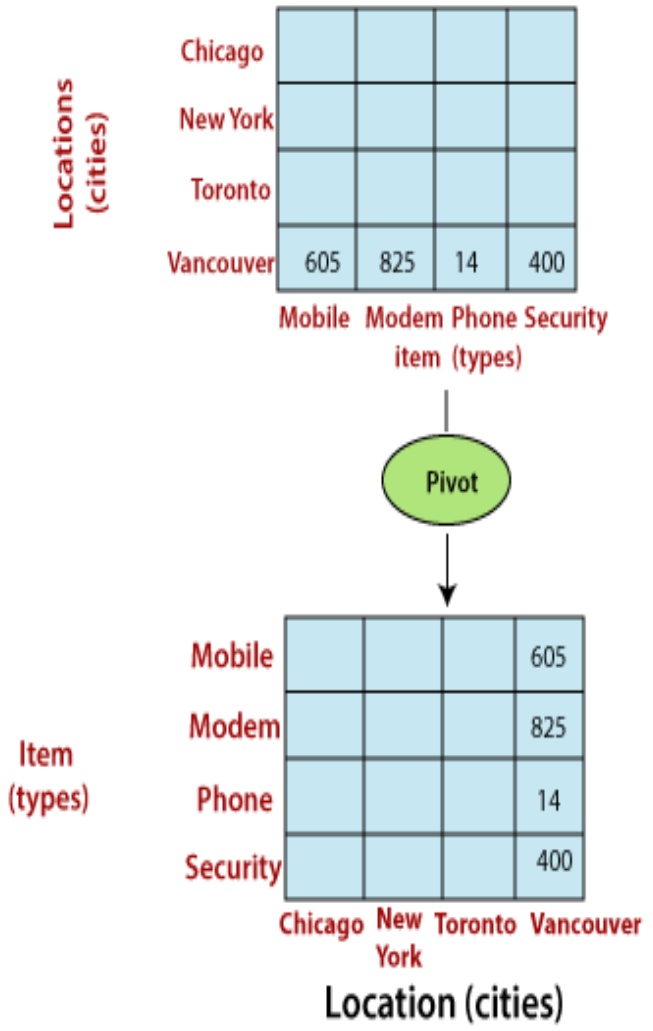
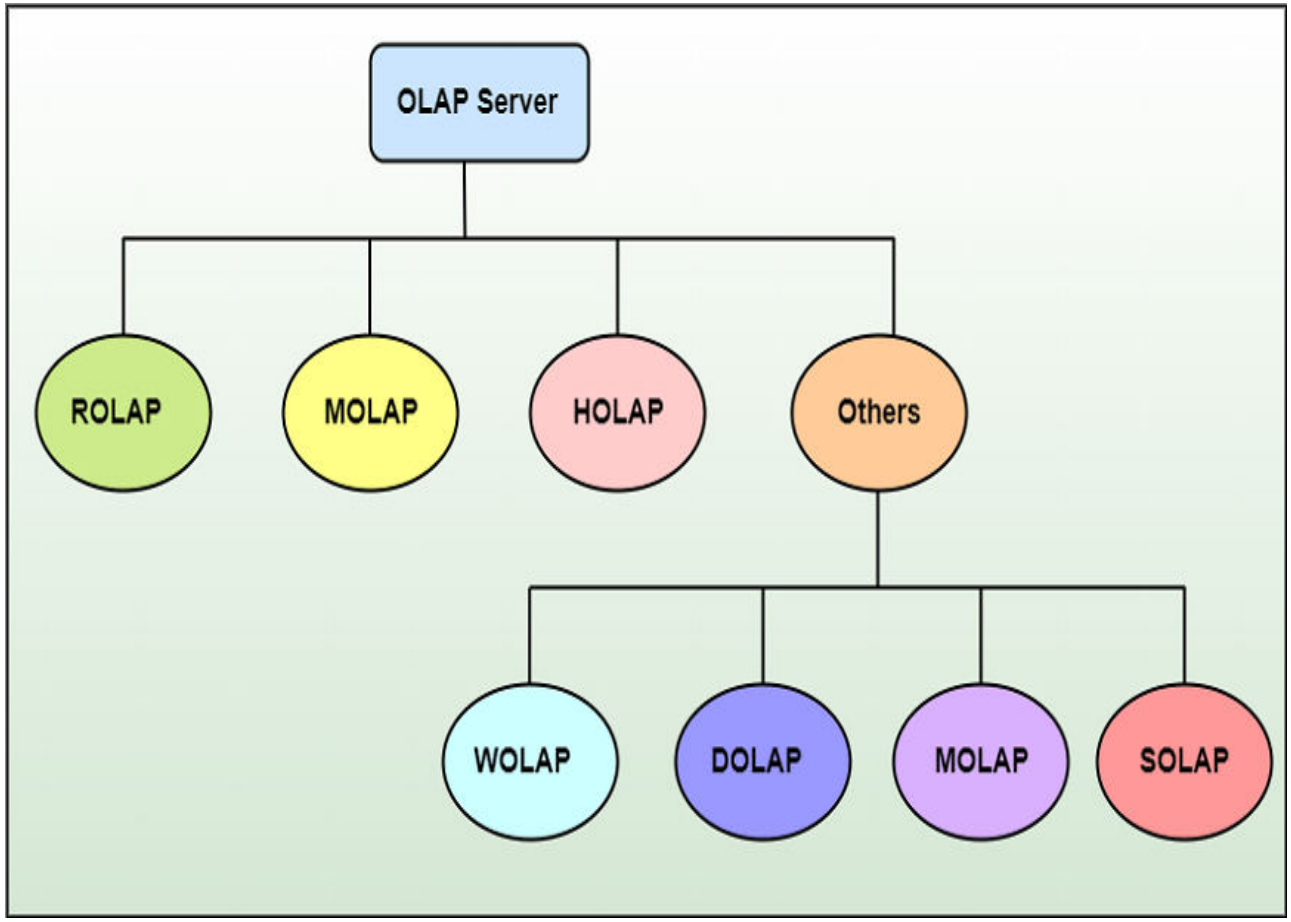


Figure: Performing pivot op

Types of OLAP Servers



Types of OLAP Servers…

We have basically three types of OLAP servers

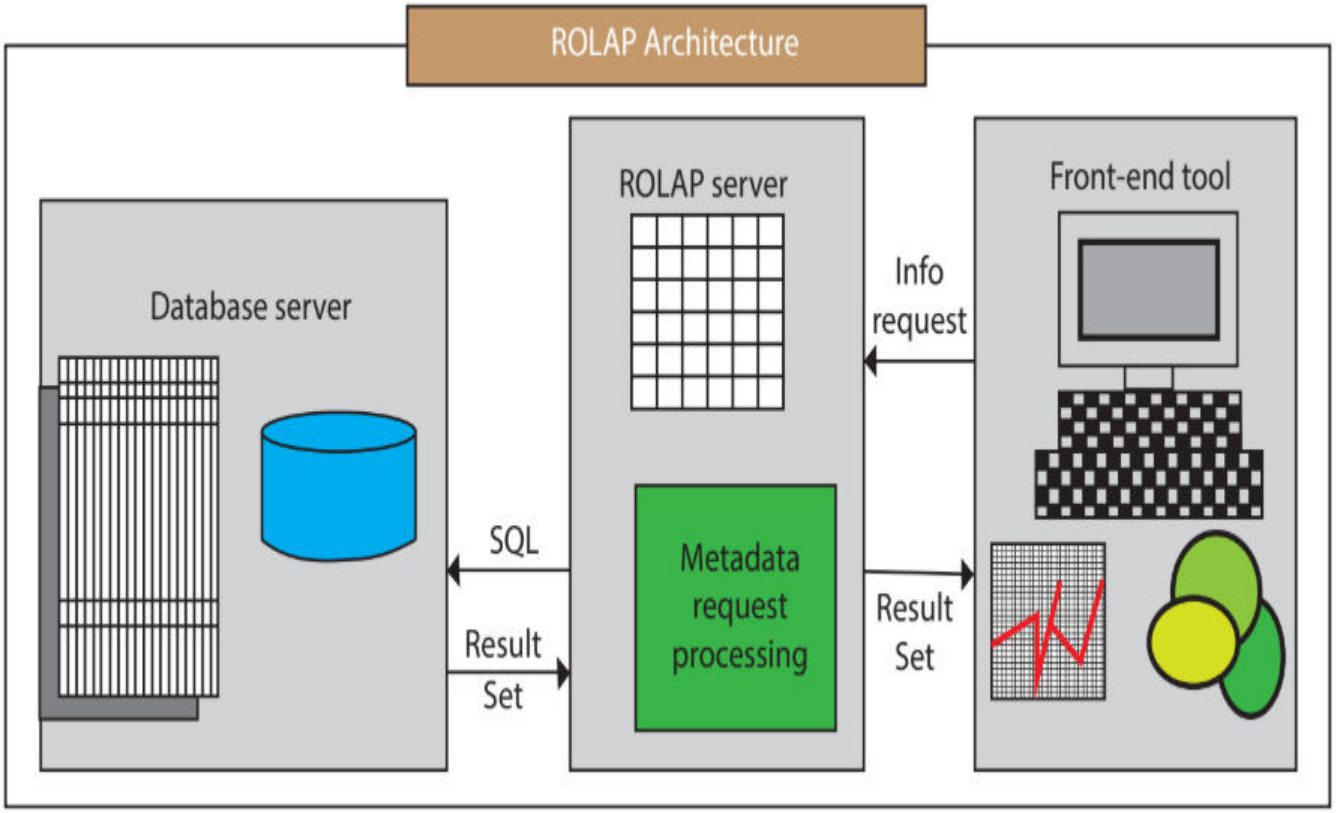
- ROLAP stands for Relational OLAP, an application based on relational DBMSs.
- MOLAP stands for Multidimensional OLAP, an application based on multidimensional DBMSs.
- HOLAP stands for Hybrid OLAP, an application using both relational and multidimensional techniques.

Types of OLAP Servers…

Relational OLAP

- Relational OLAP servers are placed between relational back-end server and client front-end tools. To store and manage the warehouse data, the relational OLAP uses relational or extended-relational DBMS. ROLAP includes the following:
- Implementation of aggregation navigation logic
- Optimization for each DBMS back-end
- Additional tools and services

Types of OLAP Servers…



Types of OLAP Servers…

- These are intermediate servers which stand in between a relational back-end server and user frontend tools.
- They use a relational or extended-relational DBMS to save and handle warehouse data, and OLAP middleware to provide missing pieces.
- ROLAP servers contain optimization for each DBMS back end, implementation of aggregation navigation logic, and additional tools and services.
- ROLAP technology tends to have higher scalability than MOLAP technology.
- ROLAP systems work primarily from the data that resides in a relational database, where the base data and dimension tables are stored as relational tables. This model permits the multidimensional analysis of data.

Types of OLAP Servers…

- This technique relies on manipulating the data stored in the relational database to give the presence of traditional OLAP's slicing and dicing functionality. In essence, each method of slicing and dicing is equivalent to adding a "WHERE" clause in the SQL statement.
- ROLAP Architecture includes the following components
  - Database server.
  - ROLAP server.
  - Front-end tool.

Types of OLAP Servers…

- ROLAP is the latest and fastest-growing OLAP technology segment in the market. This method allows multiple multidimensional views of two-dimensional relational tables to be created, avoiding structuring record around the desired view.

- Some products in this segment have supported reliable SQL engines to help the complexity of multidimensional analysis. This includes creating multiple SQL statements to handle user requests, being 'RDBMS' aware and also being capable of generating the SQL statements based on the optimizer of the DBMS engine.

Types of OLAP Servers…

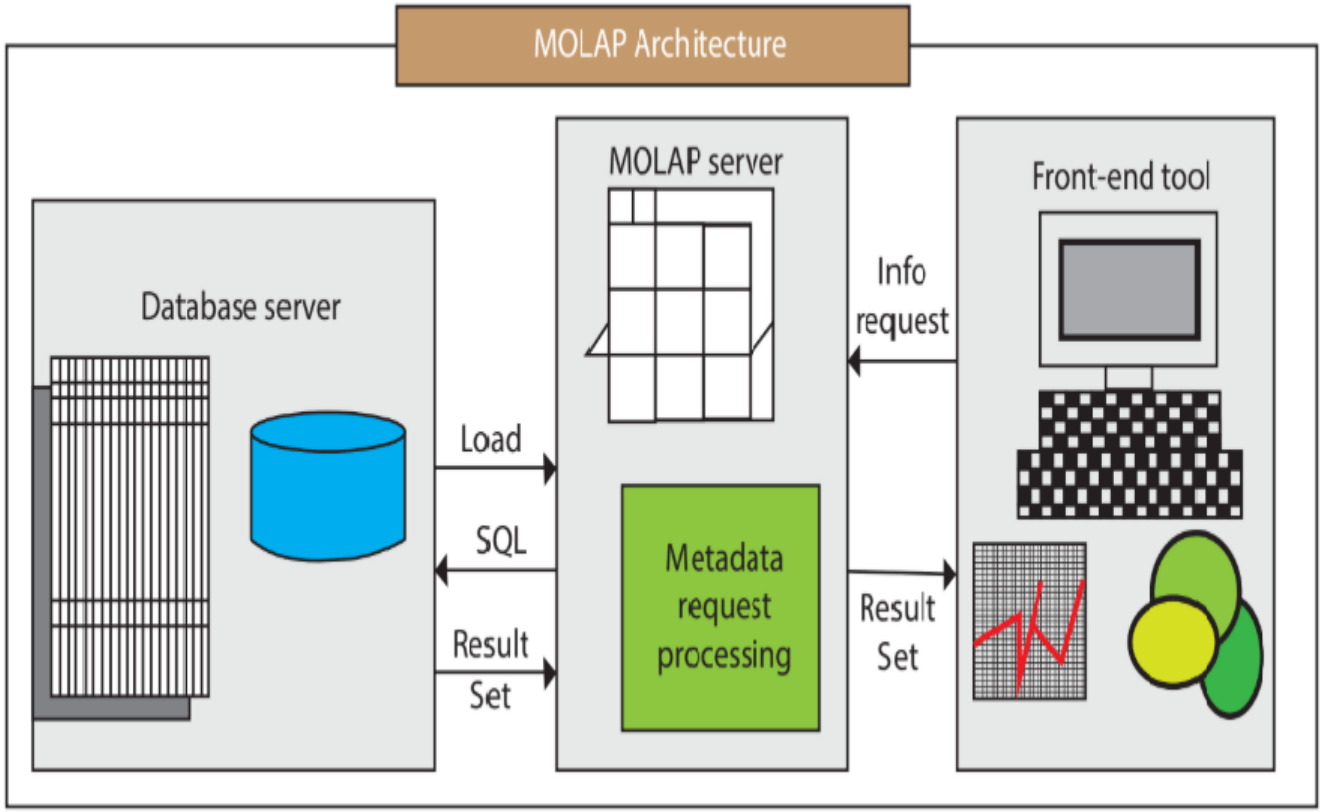
Advantages

- Can handle large amounts of information: The data size limitation of ROLAP technology is depends on the data size of the underlying RDBMS. So, ROLAP itself does not restrict the data amount.
- <="" strong="">RDBMS already comes with a lot of features. So ROLAP technologies, (works on top of the RDBMS) can control these functionalities.

Disadvantages

- Performance can be slow: Each ROLAP report is a SQL query (or multiple SQL queries) in the relational database, the query time can be prolonged if the underlying data size is large.
- Limited by SQL functionalities: ROLAP technology relies on upon developing SQL statements to query the relational database, and SQL statements do not suit all needs.
- round the desired view.

Types of OLAP Servers…



Types of OLAP Servers…

- A MOLAP system is based on a native logical model that directly supports multidimensional data and operations. Data are stored physically into multidimensional arrays, and positional techniques are used to access them.
- One of the significant distinctions of MOLAP against a ROLAP is that data are summarized and are stored in an optimized format in a multidimensional cube, instead of in a relational database. In MOLAP model, data are structured into proprietary formats by client's reporting requirements with the calculations pre-generated on the cubes.

Types of OLAP Servers…

MOLAP Architecture includes the following components

- Database server
- MOLAP server
- Front-end tool

Types of OLAP Servers…

- MOLAP structure primarily reads the precompiled data. MOLAP structure has limited capabilities to dynamically create aggregations or to evaluate results which have not been pre-calculated and stored.
- Applications requiring iterative and comprehensive time-series analysis of trends are well suited for MOLAP technology (e.g., financial analysis and budgeting).
- Examples include Arbor Software's Essbase. Oracle's Express Server, Pilot Software's Lightship Server, Sniper's TM/1. Planning Science's Gentium and Kenan Technology's Multiway.
- Some of the problems faced by clients are related to maintaining support to multiple subject areas in an RDBMS. Some vendors can solve these problems by continuing access from MOLAP tools to detailed data in and RDBMS.

Types of OLAP Servers…

- This can be very useful for organizations with performance-sensitive multidimensional analysis requirements and that have built or are in the process of building a data warehouse architecture that contains multiple subject areas.
- An example would be the creation of sales data measured by several dimensions (e.g., product and sales region) to be stored and maintained in a persistent structure.
- This structure would be provided to reduce the application overhead of performing calculations and building aggregation during initialization. These structures can be automatically refreshed at predetermined intervals established by an administrator.

Types of OLAP Servers…

Advantages

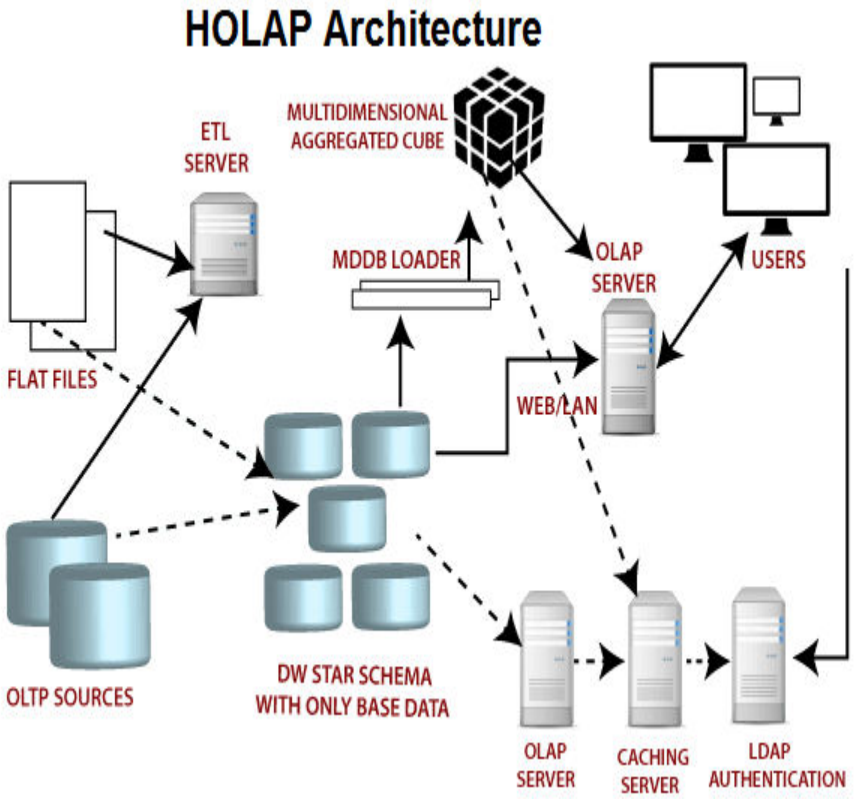
- Excellent Performance: A MOLAP cube is built for fast information retrieval, and is optimal for slicing and dicing operations.
- Can perform complex calculations: All evaluation have been pre-generated when the cube is created. Hence, complex calculations are not only possible, but they return quickly.

Types of OLAP Servers…

Disadvantages

- Limited in the amount of information it can handle: Because all calculations are performed when the cube is built, it is not possible to contain a large amount of data in the cube itself.
- Requires additional investment: Cube technology is generally proprietary and does not already exist in the organization. Therefore, to adopt MOLAP technology, chances are other investments in human and capital resources are needed.

Types of OLAP Servers…



Types of OLAP Servers…

Hybrid OLAP (HOLAP) Server

- HOLAP incorporates the best features of MOLAP and ROLAP into a single architecture. HOLAP systems save more substantial quantities of detailed data in the relational tables while the aggregations are stored in the pre-calculated cubes.
- HOLAP also can drill through from the cube down to the relational tables for delineated data. The Microsoft SQL Server 2000 provides a hybrid OLAP server.

Types of OLAP Servers…

Advantages of HOLAP

- HOLAP provide benefits of both MOLAP and ROLAP.
- It provides fast access at all levels of aggregation.
- HOLAP balances the disk space requirement, as it only stores the aggregate information
- on the OLAP server and the detail record remains in the relational database.
- So no duplicate copy of the detail record is maintained.

Disadvantages of HOLAP

- HOLAP architecture is very complicated because it supports both MOLAP and ROLAP servers.

Types of OLAP Servers…

Web-Enabled OLAP (WOLAP) Server

- WOLAP pertains to OLAP application which is accessible via the web browser. Unlike traditional client/server OLAP applications, WOLAP is considered to have a three-tiered architecture which consists of three components: a client, a middleware, and a database server.

Desktop OLAP (DOLAP) Server

- DOLAP permits a user to download a section of the data from the database or source, and work with that dataset locally, or on their desktop.

Types of OLAP Servers…

Mobile OLAP (MOLAP) Server

- Mobile OLAP enables users to access and work on OLAP data and applications remotely through the use of their mobile devices.

Spatial OLAP (SOLAP) Server

- SOLAP includes the capabilities of both Geographic Information Systems (GIS) and OLAP into a single user interface. It facilitates the management of both spatial and non-spatial data.

Computation of Data Cubes and OLAP Queries

- Data warehouses contain huge volumes of data.
- OLAP servers demand that decision support queries be answered in the order of seconds.
- Therefore, it is crucial for data warehouse systems to support highly efficient cube computation techniques, access methods, and query processing techniques.
- At the core of multidimensional data analysis is the efficient computation of aggregations across many sets of dimensions.
- In SQL terms, these aggregations are referred to as group-by's. Each group-by can be represented by *cuboids*, where the set of group-by's forms a lattice of cuboids defining a data cube.

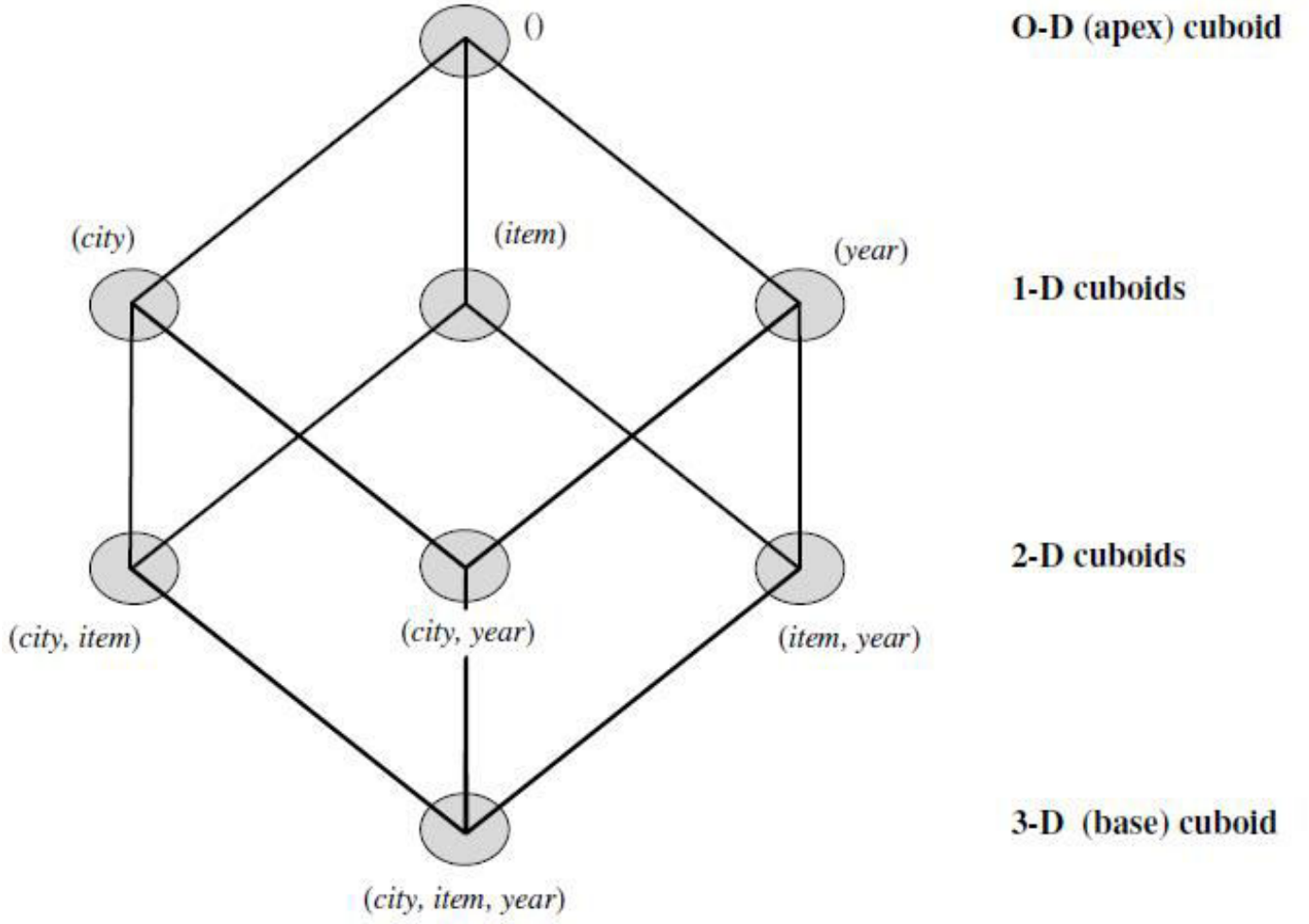
Computation of Data Cubes and OLAP Queries…

- One approach to cube computation extends SQL so as to include a compute cube operator.
- The compute cube operator computes aggregates over all subsets of the dimensions specified in the operation.
- This can require excessive storage space, especially for large numbers of dimensions.
- Suppose that you would like to create a data cube for *Electronics* sales that contains the following: *city, item, year*, and *sales in dollars*.

Computation of Data Cubes and OLAP Queries…

- Taking the three attributes, *city, item*, and *year*, as the dimensions for the data cube, and *sales in dollars* as the measure, the total number of cuboids, or group-by's, that can be computed for this data cube is $2^3 = 8$.
- The possible group-by's are the following: f(*city, item, year*), (*city, item*), (*city, year*), (*item, year*), (*city*), (*item*), (*year*), ()g, where () means that the group-by is empty (i.e., the dimensions are not grouped).
- These group-by's form a lattice of cuboids for the data cube, as shown in Figure below.

Computation of Data Cubes and OLAP Queries…



Computation of Data Cubes and OLAP Queries…

- The base cuboid contains all three dimensions, *city, item*, and *year*. It can return the total sales for any combination of the three dimensions.
- The apex cuboid, or 0-D cuboid, refers to the case where the group-by is empty. It contains the total sum of all sales.
- The base cuboid is the least generalized (most specific) of the cuboids.
- The apex cuboid is the most generalized (least specific) of the cuboids, and is often denoted as all.
- If we start at the apex cuboid and explore downward in the lattice, this is equivalent to drill down within the data cube.
- If we start at the base cuboid and explore upward, this is equivalent to roll up.

Computation of Data Cubes and OLAP Queries…

- An SQL query containing no group-by, such as “compute the sum of total sales,” is a zero-dimensional operation.
- An SQL query containing one group-by, such as “compute the sum of sales, group by city,” is a one-dimensional operation.
- A cube operator on n dimensions is equivalent to a collection of group by statements, one for each subset of the n dimensions.
- Therefore, the cube operator is the n-dimensional generalization of the group by operator. Based on the syntax of DMQL, the data cube in this example could be defined as

  define cube sales cube [city, item, year]: sum(sales in dollars)

Computation of Data Cubes and OLAP Queries…

- For a cube with *n* dimensions, there are a total of 2*n* cuboids, including the base cuboid.
- A statement such as *compute cube sales cube*

  would explicitly instruct the system to compute the sales aggregate cuboids for all of

  the eight subsets of the set {*city, item, year*}, including the empty subset.
- On-line analytical processing may need to access different cuboids for different queries.
- Therefore, it may seem like a good idea to compute all or at least some of the cuboids in a data cube in advance.
- Pre-computation leads to fast response time and avoid some redundant computation.

Computation of Data Cubes and OLAP Queries…

- A major challenge related to this pre-computation, however, is that the required storage space may explode if all of the cuboids in a data cube are pre-computed, especially when the cube has many dimensions.
- The storage requirements are even more excessive when many of the dimensions have associated concept hierarchies, each with multiple levels.
- This problem is referred to as the ***curse of dimensionality***.
- If there were no hierarchies associated with each dimension, then the total number of cuboids for an *n*-dimensional data cube, as we have seen above, is 2^n.
- However, in practice, many dimensions do have hierarchies. For example, the dimension *time* is usually not explored at only one conceptual level, such as *year*, but rather at multiple conceptual levels, such as in the hierarchy "*day* < *month* < *quarter* < *year*".

Efficient Data Cube Computation

- Data cube can be viewed as a lattice of cuboids
  - The bottom-most cuboid is the base cuboid
  - The top-most cuboid (apex) contains only one cell
  - How many cuboids in an n-dimensional cube with L levels?

$$T = \prod_{i=1}^{n} (L_i + 1)$$

- Materialization of data cube
  - Materialize every (cuboid) (full materialization), none (no materialization), or some (partial materialization)
  - Selection of which cuboids to materialize
    - Based on size, sharing, access frequency, etc.

Efficient Data Cube Computation

Example

- If the cube has 10 dimensions and each dimension has 4 levels, what will be the number of cuboids generated?

Solution

Here n=10 Li=4 for i=1,2…….10

$$T = \prod_{i=1}^{n} (L_i + 1)$$

Thus, Total number of cuboids=

$5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 = 5^{10} \approx 9.8 \times 10^{6}$

- From above example, it is unrealistic to pre-compute and materialize all of the cuboids that can possibly be generated for a data cube (or from a base cuboid). If there are many cuboids, and these cuboids are large in size, a more reasonable option is partial materialization, that is, to materialize only some of the possible cuboids that can be generated.

Efficient Data Cube Computation

Computation of Selected Cuboids

There are three choices for data cube materialization given a base cuboid:

- **No materialization:**
  - Do not pre-compute any of the cuboids.
  - This leads to computing expensive multidimensional aggregates on the fly, which can be extremely slow.
- **Full materialization:**
  - Pre-compute all of the cuboids.
  - The resulting lattice of computed cuboids is referred to as the *full cube*.
  - This choice typically requires huge amounts of memory space in order to store all of the pre-computed cuboids.

Efficient Data Cube Computation

Computation of Selected Cuboids

- **Partial materialization:**
  - Selectively compute a proper subset of the whole set of possible cuboids.
  - Alternatively, we may compute a subset of the cube, which contains only those cells that satisfy some user-specified criterion, such as where the tuple count of each cell is above some threshold.
  - We will use the term *sub-cube* to refer to the latter case, where only some of the cells may be pre-computed for various cuboids.
  - Partial materialization represents an interesting trade-off between storage space and response time.

Efficient Data Cube Computation

Computation of Selected Cuboids

- The partial materialization of cuboids or sub-cubes should consider three factors:
  - (1) identify the subset of cuboids or sub-cubes to materialize
  - (2) exploit the materialized cuboids or sub-cubes during query processing
  - (3) efficiently update the materialized cuboids or sub-cubes during load and refresh.
- The selection of the subset of cuboids or sub-cubes to materialize should take into account the queries in the workload, their frequencies, and their accessing costs.
- In addition, it should consider workload characteristics, the cost for incremental updates, and the total storage requirements.

Efficient Data Cube Computation

Computation of Selected Cuboids

- A popular approach is to materialize the set of cuboids on which other frequently referenced cuboids are based.
- Alternatively, we can compute an ***iceberg cube***, which is a data cube that stores only those cube cells whose aggregate value (e.g., count) is above some minimum support threshold.
- Another common strategy is to materialize a ***shell cube***. This involves pre-computing the cuboids for only a small number of dimensions (such as 3 to 5) of a data cube.
- Queries on additional combinations of the dimensions can be computed on-the-fly.
- An iceberg cube can be specified with an SQL query, as shown in the following example.

Iceberg Cube

- Computing only the cuboid cells whose count or other aggregates satisfying the condition like

compute cube sales iceberg as

month, city, customer group, count(*)

From salesInfo

cube by month, city, customer group

having count(*) >= min_sup

Iceberg Cube

- Motivation
  - Only a small portion of cube cells may be "above the water'' in a sparse cube
  - Only calculate "interesting" cells—data above certain threshold
  - Avoid explosive growth of the cube
    - Suppose 100 dimensions, only 1 base cell. How many aggregate cells if count >= 1? What about count >= 2?
- The compute cube statement specifies the pre-computation of the iceberg cube, *sales iceberg*, with the dimensions *month*, *city*, and *customer group*, and the aggregate measure count().
- The input tuples are in the *salesInfo* relation.
- The cube by clause specifies that aggregates (group-by's) are to be formed for each of the possible subsets of the given dimensions.

Indexing OLAP Data: Bitmap Index

- Index on a particular column
- Each value in the column has a bit vector: bit-op is fast
- The length of the bit vector: # of records in the base table
- The *i*-th bit is set if the *i*-th row of the base table has the value for the indexed column
- not suitable for high cardinality domains

Base table

Cust	Region	Type
C1	Asia	Retail
C2	Europe	Dealer
C3	Asia	Dealer
C4	America	Retail
C5	Europe	Dealer

Index on Region

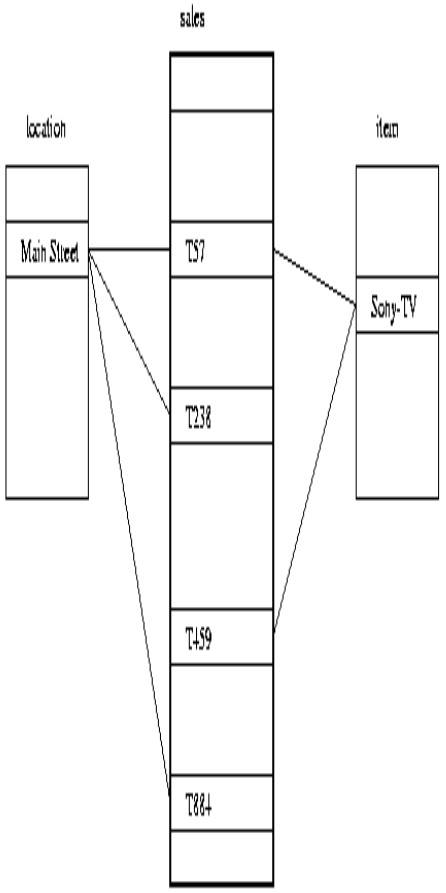
RecID	Asia	Europe	America
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1
5	0	1	0

Index on Type

RecID	Retail	Dealer
1	1	0
2	0	1
3	0	1
4	1	0
5	0	1

Indexing OLAP Data: Join Indices

- Join index: JI(R-id, S-id) where R (R-id, …) ▷◁ S (S-id, …)
- Traditional indices map the values to a list of record ids
  - It materializes relational join in JI file and speeds up relational join
- In data warehouses, join index relates the values of the dimensions of a star schema to rows in the fact table.
  - E.g. fact table: *Sales* and two dimensions *city* and *product*
    - A join index on *city* maintains for each distinct city a list of R-IDs of the tuples recording the Sales in the city
  - Join indices can span multiple dimensions



Data Warehouse Back end Tools

Data Cleaning

- The data warehouse involves large volumes of data from multiple sources, which can lead to a high probability of errors and anomalies in the data. Inconsistent field lengths, inconsistent descriptions, inconsistent value assignments, missing entries and violation of integrity constraints are some of the examples.
- The three classes of data cleaning tools are popularly used to help detect data anomalies and correct them:
  - **Data migration tools** allow simple transformation rules to be specified.
  - **Data scrubbing tools** use domain-specific knowledge to do the scrubbing of data. Tools such as Integrity fall in this category.
  - **Data auditing tools** make it possible to discover rules and relationships by scanning data. Thus, such tools may be considered variants of data mining tools.

Data Warehouse Back end Tools

Load

- After extracting, cleaning, and transforming, data will be loaded into the data warehouse.
- A load utility has to allow the system administrator to monitor status, to cancel, suspend and resume a load, and to restart after failure with no loss of data integrity.
- Sequential loads can take a very long time to complete especially when it deals with terabytes of data.
- Therefore, pipelined and partitioned parallelism are typically used. Also incremental loading over full load is more popularly used with most commercial utilities since it reduces the volume of data that has to be incorporated into the data warehouse.

Data Warehouse Back end Tools

Refresh

- Refreshing a warehouse consists in propagating updates on source data to correspondingly update the base data and derived data stored in the warehouse.
- There are two sets of issues to consider: when to refresh, and how to refresh. Usually, the warehouse is refreshed periodically (e.g., daily or weekly).
- Only if some OLAP queries need current data, it is necessary to propagate every update.
- The refresh policy is set by the warehouse administrator, depending on user needs and may be different for different sources.

Data Warehouse Back end Tools

- Refresh techniques may also depend on the characteristics of the source and the capabilities of the database servers.
- Extracting an entire source file or database is usually too expensive, but may be the only choice for legacy data sources.
- Most contemporary database systems provide replication servers that support incremental techniques for propagating updates from a primary database to one or more replicas.
- Such replication servers can be used to incrementally refresh a warehouse when the sources change.

Data Warehouse Tuning

- The process of applying different strategies in performing different operations of data warehouse such that performance measures will enhance is called data warehousing tuning.
- For this, it is very important to have a complete knowledge of data warehouse.
- We can tune the different aspects of a data warehouse such as performance, data load, queries, etc.
- A data warehouse keeps evolving and it is unpredictable what query the user is going to post in the future.
- Therefore it becomes more difficult to tune a data warehouse system.

Data Warehouse Tuning…

Tuning a data warehouse is a difficult procedure due to following reasons:

- Data warehouse is dynamic; it never remains constant.
- It is very difficult to predict what query the user is going to post in the future.
- Business requirements change with time.
- Users and their profiles keep changing.
- The user can switch from one group to another.
- The data load on the warehouse also changes with time.

Data Warehouse - Testing

Testing is very important for data warehouse systems to make them work correctly and efficiently.

There are three basic levels of testing performed on a data warehouse:

- **Unit testing**
- **Integration testing**
- **System testing**

Data Warehouse – Testing…

Unit Testing

- In unit testing, each component is separately tested.
- Each module, i.e., procedure, program, SQL Script, Unix shell is tested.
- This test is performed by the developer.

Integration Testing

- In integration testing, the various modules of the application are brought together and then tested against the number of inputs.
- It is performed to test whether the various components do well after integration.

Data Warehouse – Testing…

System Testing

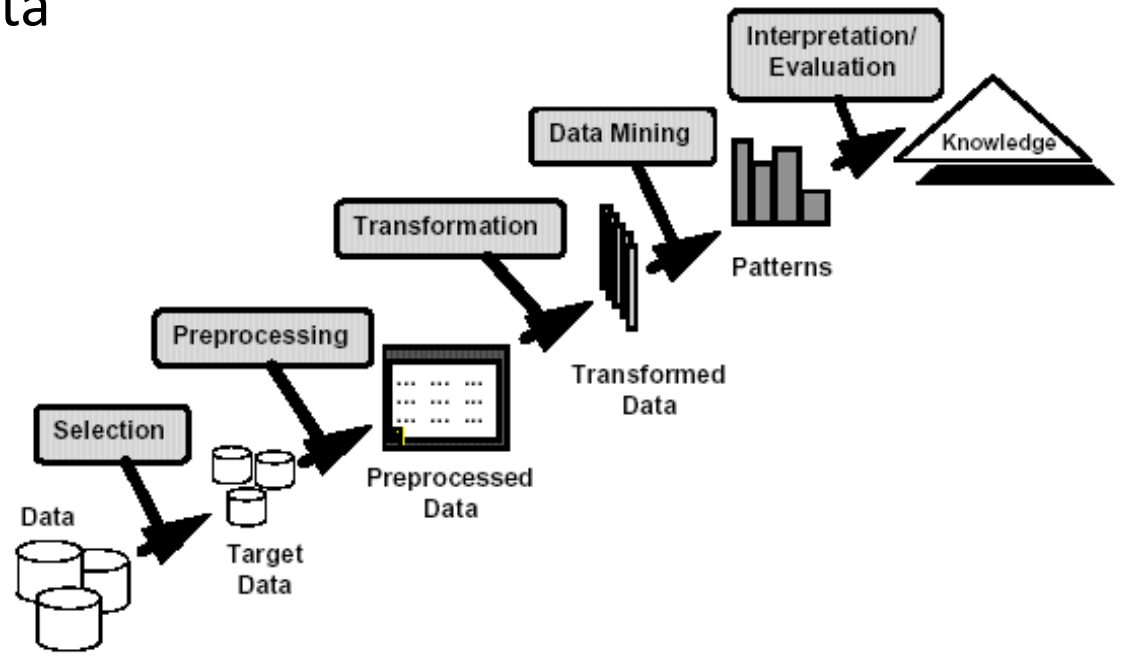
- In system testing, the whole data warehouse application is tested together.
- The purpose of system testing is to check whether the entire system works correctly together or not.
- System testing is performed by the testing team.
- Since the size of the whole data warehouse is very large, it is usually possible to perform minimal system testing before the test plan can be enacted.

DWDM

Unit 5.1

What is Data Mining?

- Many Definitions
  - Non-trivial extraction of implicit, previously unknown and potentially useful information from data
  - Exploration & analysis, by automatic or semi-automatic means, of large quantities of data in order to discover meaningful patterns



What is (not) Data Mining?

- What is not Data Mining? (Trivial)
  - Look up phone number in phone directory
  - Query a Web search engine for information about “Amazon”

- What is Data Mining? (Non Trivial)
  - Certain names are more prevalent in certain New Road locations [classification]
  - Group together similar documents returned by search engine according to their context (e.g. Amazon rainforest, Amazon.com) [clustering]

Why Mine Data? Commercial Viewpoint

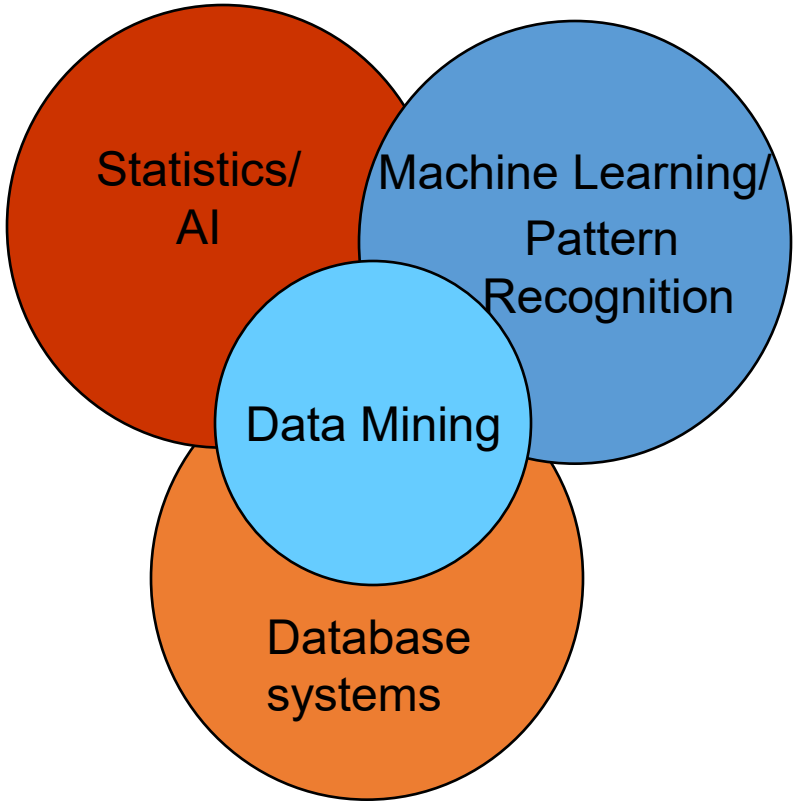
- Lots of data is being collected and warehoused
  - Web data, e-commerce
  - purchases at department/ grocery stores
  - Bank/Credit Card transactions
- There is often information “hidden” in the data that is not readily evident
- Human analysts may take weeks to discover useful information
- Much of the data is never analyzed at all

Why Mine Data? Scientific Viewpoint

- Data collected and stored at enormous speeds (GB/hour)
- Traditional techniques infeasible for raw data
- Data mining may help scientists
  - in classifying and segmenting data
  - in Hypothesis Formation

Origins of Data Mining

- **Draws ideas from machine learning/AI, pattern recognition, statistics, and database systems**
- **Traditional Techniques may be unsuitable due to**
  - **Enormity of data**
  - **High dimensionality of data**
  - **Heterogeneous, distributed nature of data**
  - **Velocity (faster generation of data)**



Data Mining Tasks (Functionalities)

- Concept/Class Description
- Mining Frequent Patterns, Associations, and Correlations
- Classification
- Prediction
- Cluster Analysis
- Outlier Analysis
- Evolution Analysis

Data Mining Tasks (Functionalities)...

- **Concept/Class Description**
  - These descriptions can be derived via data characterization or data discrimination.
    - Data characterization: It is a summarization of the general characteristics or features of a target class of data. Example: summarizing the characteristics of customers who spend more than $5,000 in a month.
    - Data discrimination: It is a comparison of the general features of target class data objects with the general features of objects from one or a set of contrasting classes. Example: Buying Customer Vs. Non Buying Customer.

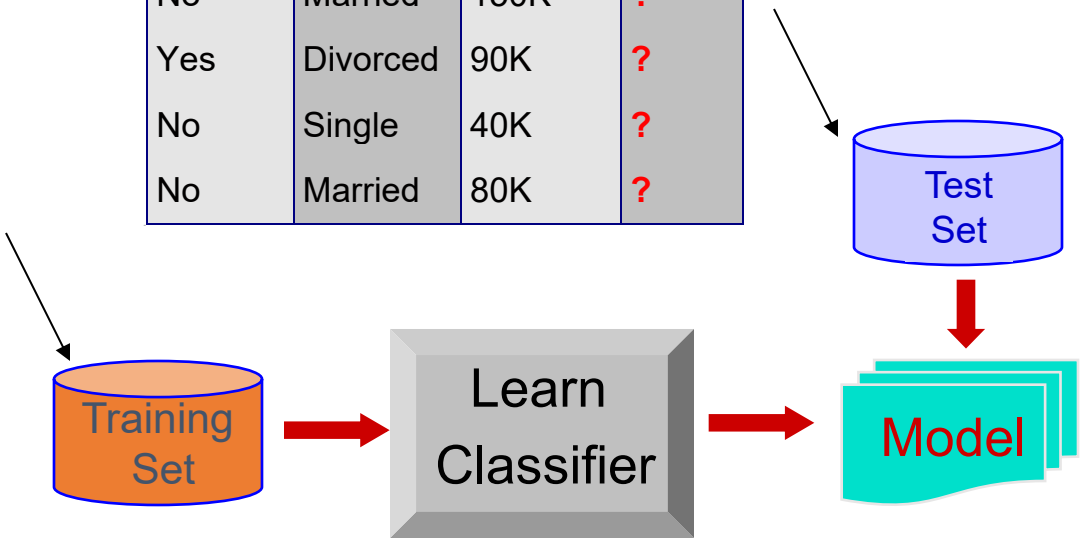
Classification: Definition

- Given a collection of records (*training set*), each record containing a set of *attributes* where one of the attributes is the *class*, find a *model* for class attribute as a function of the values of other attributes.
- Goal: previously unseen records should be assigned a class as accurately as possible.
  - A *test set* is used to determine the accuracy of the model. Usually, the given data set is divided into training and test sets, with training set used to build the model and test set used to validate it.

Classification Example

Tid	Refund (categorical)	Marital Status (categorical)	Taxable Income (continuous)	Cheat (class)
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Refund	Marital Status	Taxable Income	Cheat
No	Single	75K	?
Yes	Married	50K	?
No	Married	150K	?
Yes	Divorced	90K	?
No	Single	40K	?
No	Married	80K	?



Classification: Application 1

- **Direct Marketing**
  - Goal: Reduce cost of mailing by *targeting* a set of consumers likely to buy a new cell-phone product.
  - Approach:
    - Use the data for a similar product introduced before.
    - We know which customers decided to buy and which decided otherwise. This *{buy, don't buy}* decision forms the *class attribute*.
    - Collect various demographic, lifestyle, and company-interaction related information about all such customers.
      - Type of business, where they stay, how much they earn, etc.
    - Use this information as input attributes to learn a classifier model.

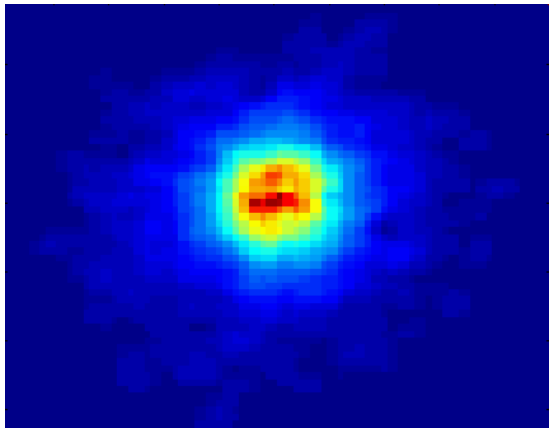
Classification: Application 2

- **Fraud Detection**
  - Goal: Predict fraudulent cases in credit card transactions.
  - Approach:
    - Use credit card transactions and the information on its account-holder as attributes.
      - When does a customer buy, what does he buy, how often he pays on time, etc.
    - Label past transactions as fraud or fair transactions. This forms the class attribute.
    - Learn a model for the class of the transactions.
    - Use this model to detect fraud by observing credit card transactions on an account.

Classification: Application 3

- Galaxy Classification

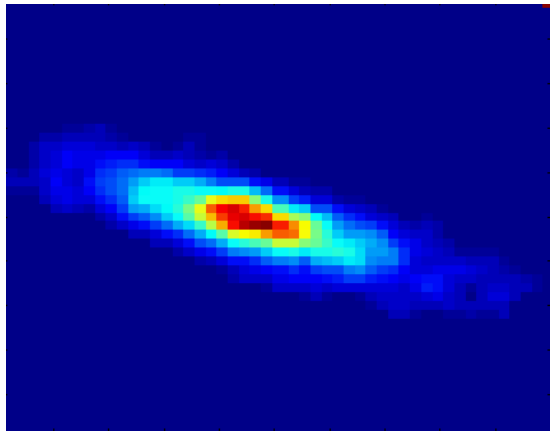
Early



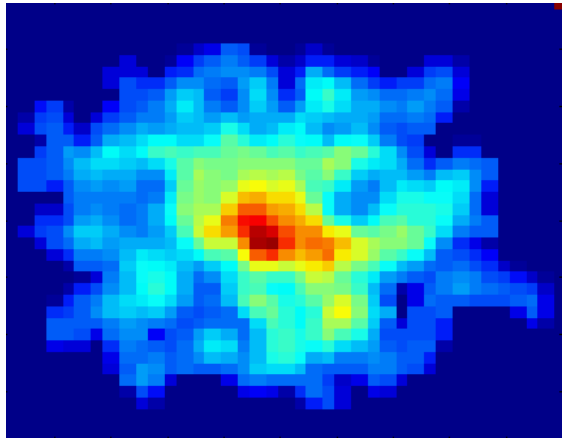
Class:
- Stages of Formation

Attributes:
- Image features,
- Characteristics of light waves received, etc.

Intermediate



Late



Data Size:
- 72 million stars, 20 million galaxies
- Object Catalog: 9 GB
- Image Database: 150 GB

Clustering Definition

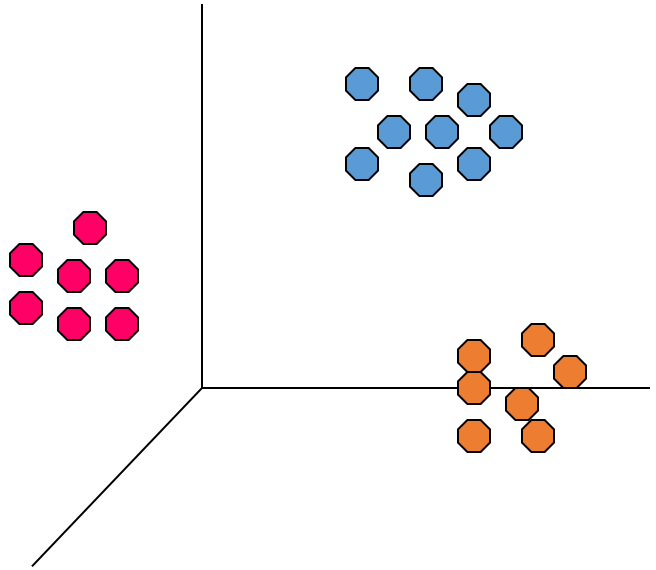
- Given a set of data points, each having a set of attributes, and a similarity measure among them, find clusters such that
  - Data points in one cluster are more similar to one another.
  - Data points in separate clusters are less similar to one another.
- Similarity Measures:
  - Euclidean Distance if attributes are continuous.
  - Other Problem-specific Measures.

Illustrating Clustering

Euclidean Distance Based Clustering in 3-D space.

Intracluster distances are minimized

Intercluster distances are maximized



Clustering: Application 1

- **Market Segmentation:**
  - Goal: subdivide a market into distinct subsets of customers where any subset may conceivably be selected as a market target to be reached with a distinct marketing mix.
  - Approach:
    - Collect different attributes of customers based on their geographical and lifestyle related information.
    - Find clusters of similar customers.
    - Measure the clustering quality by observing buying patterns of customers in same cluster vs. those from different clusters.

Clustering: Application 2

- **Document Clustering:**
  - Goal: To find groups of documents that are similar to each other based on the important terms appearing in them.

Illustrating Document Clustering

- Clustering Points: 3204 Articles of HimalyanTimes.
- Similarity Measure: How many words are common in these documents (after some word filtering).

Category	*Total Articles*	*Correctly Placed*
Financial	555	364
Foreign	341	260
National	273	36
Metro	943	746
Sports	738	573
Entertainment	354	278

Clustering of Stock Data

- Observe Stock Movements every day.
- Clustering points: Stock-{UP/DOWN}
- Similarity Measure: Two points are more similar if the events described by them frequently happen together on the same day.

Association Rule Discovery: Definition

- Given a set of records each of which contain some number of items from a given collection;
  - Produce dependency rules which will predict occurrence of an item based on occurrences of other items.

TID	*Items*
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Rules Discovered:
{Milk} --> {Coke}
{Diaper, Milk} --> {Beer}

Association Rule Discovery: Application 1

- **Marketing and Sales Promotion:**
  - Let the rule discovered be

    {Bagels, … } --> {Potato Chips}
  - Potato Chips as consequent => Can be used to determine what should be done to boost its sales.
  - Bagels in the antecedent => Can be used to see which products would be affected if the store discontinues selling bagels.
  - Bagels in antecedent *and* Potato chips in consequent => Can be used to see what products should be sold with Bagels to promote sale of Potato chips!

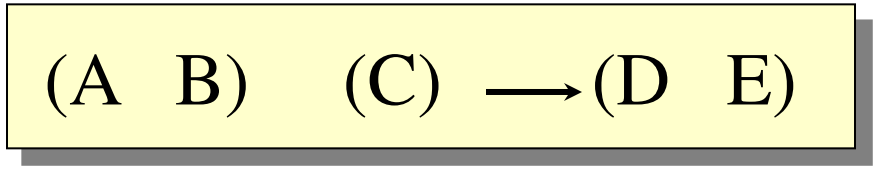
Association Rule Discovery: Application 2

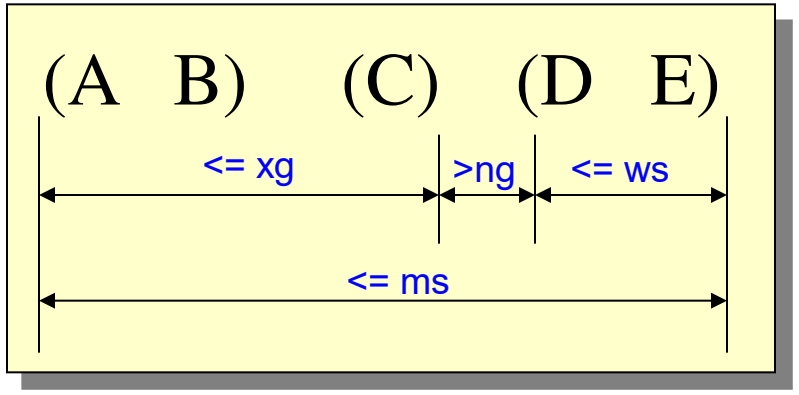
- **Supermarket shelf management:**
  - Goal: To identify items that are bought together by sufficiently many customers.
  - Approach: Process the point-of-sale data collected with barcode scanners to find dependencies among items.
  - A classic rule --
    - If a customer buys diaper and milk, then he is very likely to buy beer.
    - So, don’t be surprised if you find six-packs stacked next to diapers!

Sequential Pattern Discovery: Definition

- Given is a set of objects, with each object associated with its own timeline of events, find rules that predict strong sequential dependencies among different events.

- Rules are formed by first discovering patterns. Event occurrences in the patterns are governed by timing constraints.





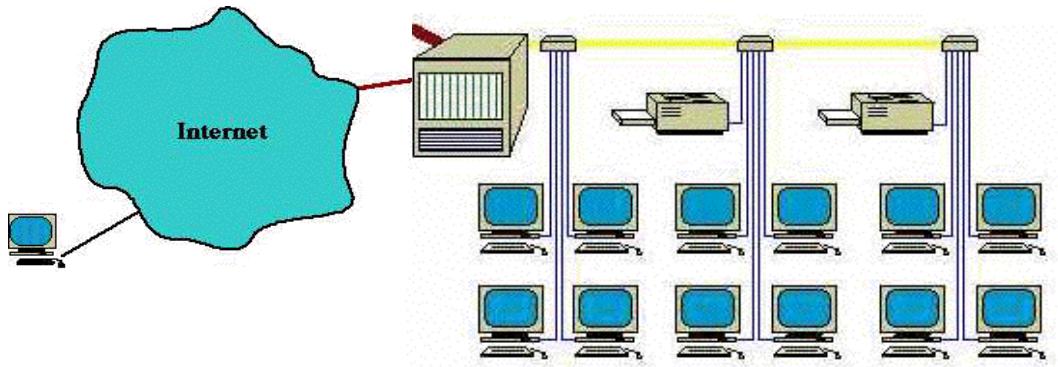
Regression

- Predict a value of a given continuous valued variable based on the values of other variables, assuming a linear or nonlinear model of dependency.
- Greatly studied in statistics, neural network fields.
- Examples:
  - Predicting sales amounts of new product based on advertising expenditure.
  - Predicting wind velocities as a function of temperature, humidity, air pressure, etc.
  - Time series prediction of stock market indices.

Deviation/Anomaly Detection

- Detect significant deviations from normal behavior

- Applications:
  - Credit Card Fraud Detection
  - Network Intrusion Detection



Outlier Analysis

- A database may contain data objects that do not comply with the general behavior or model of the data.
- These data objects are outliers.

Example: fraudulent usage of credit cards

Evolution Analysis

- Data evolution analysis describes and models regularities or trends for objects whose behavior changes over time.

  Example: stock market (time-series) data

Challenges of Data Mining

- Developing a unifying theory of data mining
- Scaling up for high dimensional data and high speed data streams
- Mining sequence data and time series data
- Mining complex knowledge from complex data
- Data mining in a network setting
- Distributed data mining and mining multi-agent data
- Data mining for biological and environmental problems
- Data Mining process-related problems
- Security, privacy and data integrity
- Dealing with non-static, unbalanced and cost-sensitive data

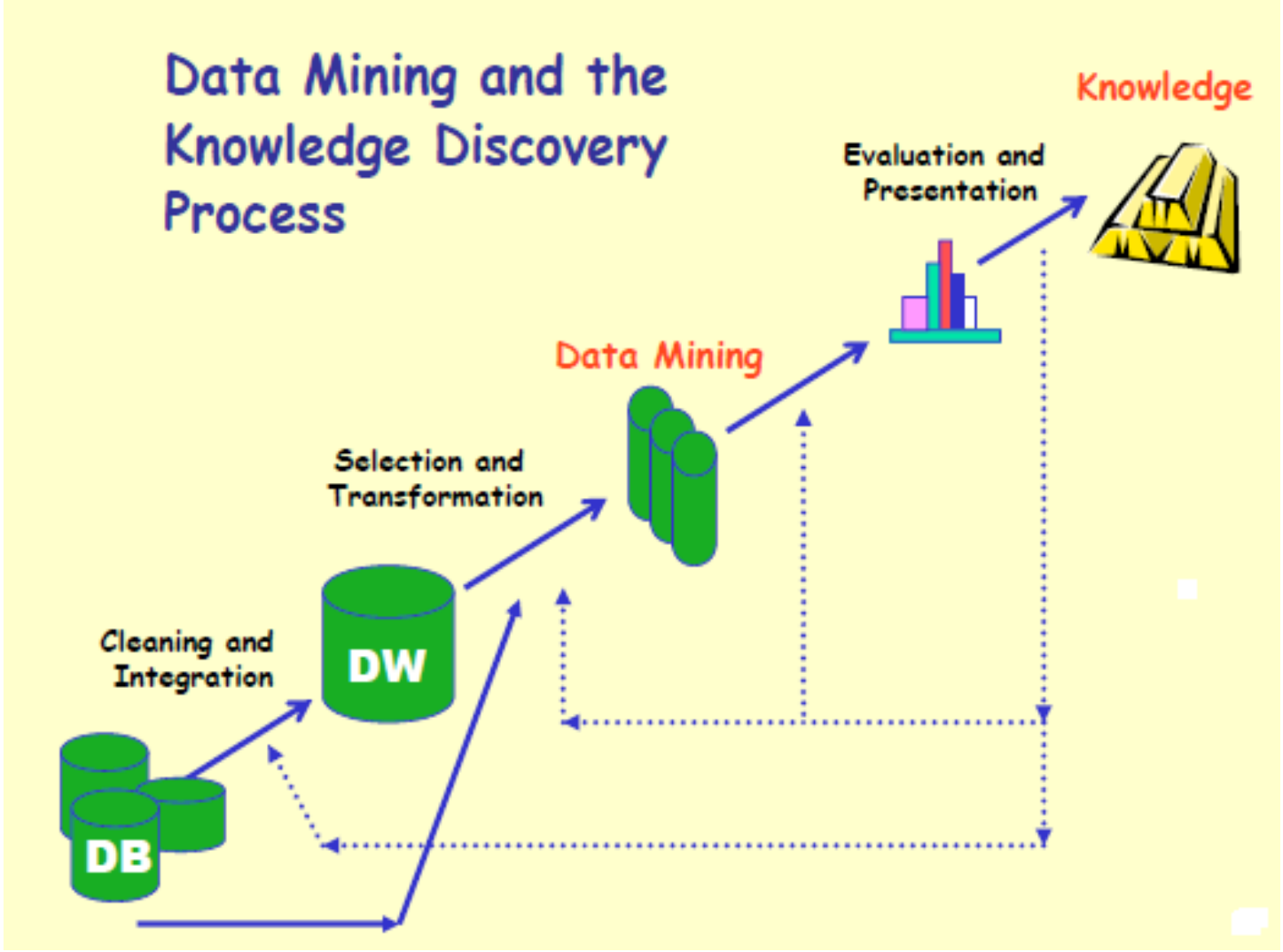
Issues in Data Mining

- **Mining methodology and user interaction issues:**
  - Mining different kinds of knowledge in databases
  - Interactive mining of knowledge at multiple levels of abstraction
  - Incorporation of background knowledge
  - Handling noisy or incomplete data
  - Pattern evaluation
- **Performance Issues:**
  - Efficiency and scalability of data mining algorithms
  - Parallel, distributed, and incremental mining algorithms

Issues in Data Mining

- Database diversity issues:
  - Handling of relational and complex types of data
  - Mining information from heterogeneous databases and global information systems

Stages of Knowledge Discovery in Database (KDD)



Stages of Knowledge Discovery in Database (KDD)…

- **Data cleaning**: It removes noise and inconsistent data
- **Data integration:** This combines data from multiple data sources
- **Data selection:** Data relevant to the analysis task are retrieved from the database
- **Data transformation:** Data are transformed or consolidated into forms appropriate for mining by performing summary or aggregation operations.

Stages of Knowledge Discovery in Database (KDD)…

- **Data mining:** an essential process where intelligent methods are applied in order to extract data patterns

- **Pattern evaluation:** Identifies the truly interesting patterns representing knowledge based on some interestingness measures.

- **Knowledge presentation:** Knowledge representation techniques are used to present the mined knowledge to the user.

Setting up KDD environment

- **Support for Extremely Large Data Set**: Data mining deals with billions of records. Thus fast and flexible way of storing and handling such large volume of data is required along with capacity of storing intermediate results.

- **Support Hybrid Learning:** Learning can be divided into three categories: Classification, Knowledge engineering, & problem solving. Complex data mining projects needs hybrid learning algorithms that possess capabilities of all three categories.

Setting up KDD environment…

- **Establish a Data Warehouse**: Data mining mainly depends upon availability and analysis of historic data and hence establishing data warehouse is important for this.

- **Introduce Data Cleaning Facility**: If data stored in databases contains noise, data processing suffers from pollution. Thus data cleaning facility is necessary to avoid such pollution.

Setting up KDD environment…

- **Facilitate Working with Dynamic Coding:** A KDD Environment should enable the user to experiment with different coding schemes. It must keep the track of genealogy of different samples and tables as well as the semantics and transformations of the different attributes are vital.

- **Beware of False Predictors:** If the results are too good to be true, you probably have found false predictors.

- **Verify Result:** Examine the results carefully and repeat and refine the knowledge discovery process until you are confident.

Application of Data Mining

- **Market Analysis and Management:** Target marketing, customer relation management, market basket analysis, cross selling, market segmentation, Find clusters of customers who share the same characteristics: interest, income level, spending habits, etc. Determine customer purchasing patterns over time

- **Risk Analysis and Management:** Forecasting, customer retention, improved underwriting, quality control, competitive analysis, credit scoring.

Application of Data Mining…

- **Internet Web Surf-Aid:** Surf-Aid applies data mining algorithms to Web access logs for market-related pages to discover customer preference and behavior pages, analyzing effectiveness of Web marketing, improving Web site organization.

- **Social Web and Networks:** User-centric applications like blogs, wikis and Web communities generate a lot of structured and semi-structured information where data mining can be used to explain and predict the evolution of social networks, personalized search for social interaction, user behavior prediction etc.

Application of Data Mining…

- **Fraud Detection and Management:** Use historical data to build models of fraudulent behavior and use data mining to help identify similar instances. For example, detect suspicious money transactions.

- **Sports:** Data mining can be used to analyze shots & fouls of different athletes, their weaknesses and helps athletes to assist in improving their games.

- **Space Science:** Data mining can be used to automate the analysis image data collected from sky survey with better accuracy.

DWDM

Unit 5.2

Contents

- Data mining primitives: What defines a data mining task?
- Design graphical user interfaces based on a data mining query language
- Architecture of data mining systems
- Summary

Why Data Mining Primitives and Languages?

- Finding all the patterns autonomously in a database?
  - unrealistic because the patterns could be too many but uninteresting
- Data mining should be an interactive process
  - User directs what to be mined
- Users must be provided with a set of primitives to be used to communicate with the data mining system
- Incorporating these primitives in a data mining query language
  - More flexible user interaction
  - Foundation for design of graphical user interface
  - Standardization of data mining industry and practice

What Defines a Data Mining Task ?

- Task-relevant data
- Type of knowledge to be mined
- Background knowledge
- Pattern interestingness measurements
- Visualization of discovered patterns

What Defines a Data Mining Task ?

- Task-relevant data
  - Typically interested in only a subset of the entire database
  - Specify
    - the name of database/data warehouse (store_db)
    - names of tables/data cubes containing relevant data (item, customer, purchases, items_sold)
    - conditions for selecting the relevant data (purchases made in Sohrakhutte for relevant year)
    - relevant attributes or dimensions (name and price from item, income and age from customer)

What Defines a Data Mining Task ? (continued)

- Type of knowledge to be mined
  - Concept description, association, classification, prediction, clustering, and evolution analysis
    - Studying buying habits of customers, mine associations between customer profile and the items they like to buy
      - Use this info to recommend items to put on sale to increase revenue
    - Studying real estate transactions, mine clusters to determine house characteristics that make for fast sales
      - Use this info to make recommendations to house sellers who want/need to sell their house quickly
    - Study relationship between individual's sport statistics and salary
      - Use this info to help sports agents and sports team owners negotiate an individual's salary

What Defines a Data Mining Task ? (continued)

- Type of knowledge to be mined
  - Pattern templates that all discovered patterns must match
  - P(X:Customer, W) and Q(X, Y) => buys(X, Z)
    - X is key of customer relation
    - P & Q are predicate variables, instantiated to relevant attributes
    - W & Z are object variables that can take on the value of their respective predicates
  - Search for association rules is confined to those matching some set of rules, such as:
    - Age(X, “30..39”) & income (X, “40K..49K”) => buys (X, “VCR”) [2.2%, 60%]
    - Customers in their thirties, with an annual income of 40-49K, are likely (with 60% confidence) to purchase a VCR, and such cases represent about 2.2% of the total number of transactions

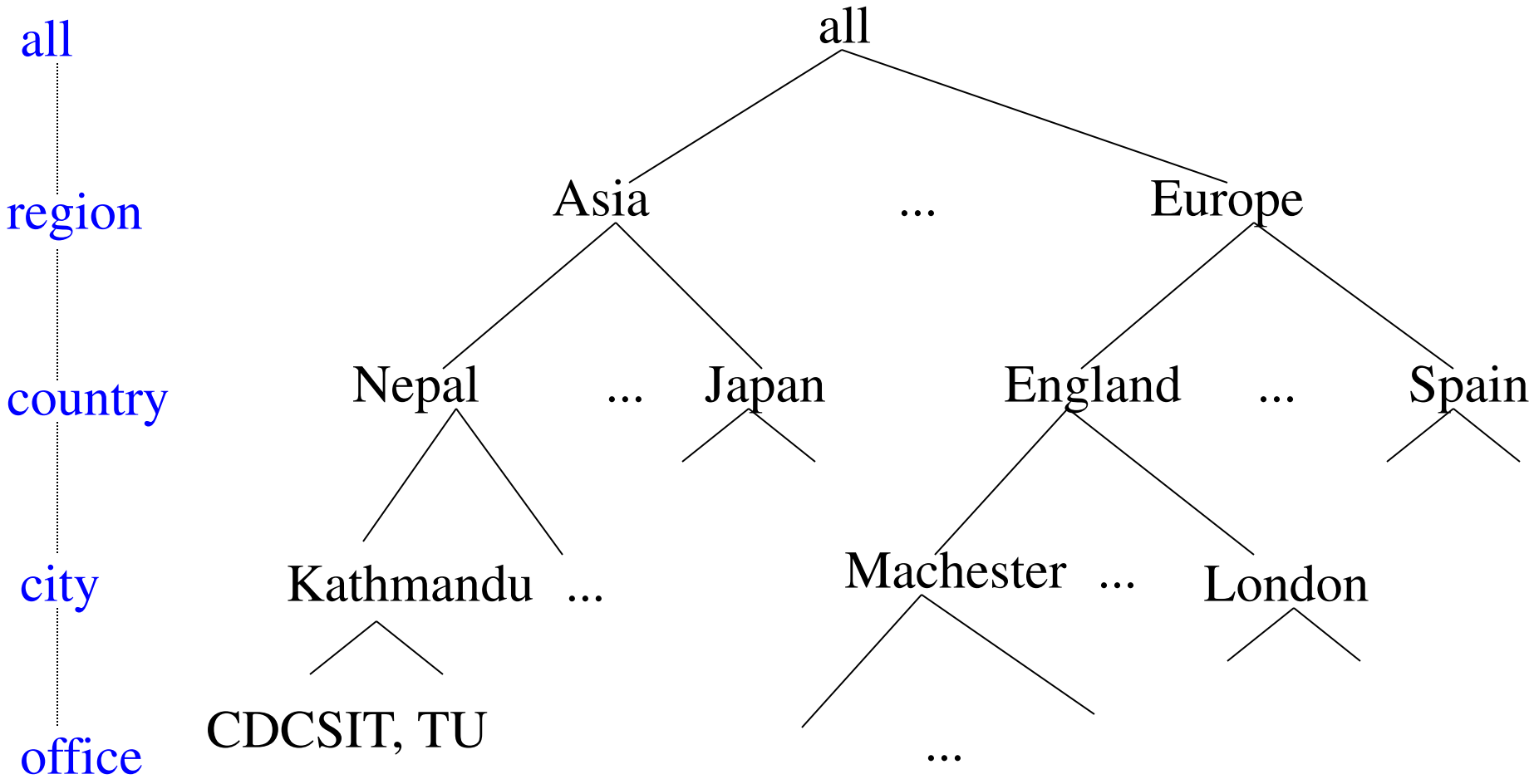
Types of knowledge to be mined

- Characterization
- Discrimination
- Association
- Classification/prediction
- Clustering
- Outlier analysis
- Other data mining tasks

Background Knowledge: Concept Hierarchies

- Allow discovery of knowledge at multiple levels of abstraction
- Represented as a set of nodes organized in a tree
  - Each node represents a concept
  - Special node, all, reserved for root of tree
- Concept hierarchies allow raw data to be handled at a higher, more generalized level of abstraction
- Four major types of concept hierarchies, schema, set-grouping, operation derived, rule based

A Concept Hierarchy: Dimension (location)



Define a sequence of mappings from a set of low level concepts to higher-level, more general concepts

Background Knowledge: Concept Hierarchies

- Schema hierarchy – total or partial order among attributes in the database schema, formally expresses existing semantic relationships between attributes
  - Table address
    - create table address (street char (50), city char (30), province_or_state char (30), country char (40));
  - Concept hierarchy location
    - street < city < province_or_state < country
- Set-grouping hierarchy – organizes values for a given attribute or dimension into groups or constant range values
  - {young, middle_aged, senior} subset of all(age)
    - {20-39} = young
    - {40-59} = middle_aged
    - {60-89} = senior

Background Knowledge: Concept Hierarchies

- Operation-derived hierarchy – based on operations specified by users, experts, or the data mining system
  - email address or a URL contains hierarchy info relating departments, universities (or companies) and countries
  - E-mail address
    - dmbook@cs.sfu.ca
  - Partial concept hierarchy
    - login-name < department < university < country

Background Knowledge: Concept Hierarchies

- Rule-based hierarchy – either a whole concept hierarchy or a portion of it is defined by a set of rules and is evaluated dynamically based on the current data and rule definition
  - Following rules used to categorize items as low profit margin, medium profit margin and high profit margin
    - Low profit margin - < $50
    - Medium profit margin – between $50 & $250
    - High profit margin - > $250
  - Rule based concept hierarchy
    - low_profit_margin (X) <= price(X, P1) and cost (X, P2) and (P1 - P2) < $50
    - medium_profit_margin (X) <= price(X, P1) and cost (X, P2) and (P1 - P2) >= $50 and (P1 – P2) <= $250
    - high_profit_margin (X) <= price(X, P1) and cost (X, P2) and (P1 - P2) > $250

Measurements of Pattern Interestingness

- After specification of task relevant data and kind of knowledge to be mined, data mining process may still generate a large number of patterns
- Typically, only a small portion of these patterns will actually be of interest to a user
- The user needs to further confine the number of uninteresting patterns returned by the data mining process
  - Utilize interesting measures
- Four types: simplicity, certainty, utility, novelty

Measurements of Pattern Interestingness (continued)

- Simplicity – A factor contributing to interestingness of pattern is overall simplicity for comprehension
  - Objective measures viewed as functions of the pattern structure or number of attributes or operators
  - More complex a rule, more difficult it is to interpret, thus less interesting
  - Example measures: rule length or number of leaves in a decision tree
- Certainty – Measure of certainty associated with pattern that assesses validity or trustworthiness
  - Confidence (A=>B) = # tuples containing both A & B/ #tuples containing A
  - Confidence of 85% for association rule buys (X, computer) => buys (X, software) means 85% of all customers who bought a computer bought software also

Measurements of Pattern Interestingness (continued)

- Utility – potential usefulness of a pattern is a factor determining its interestingness
  - Estimated by a utility function such as support – percentage of task relevant data tuples for which pattern is true
    - Support (A=>B) = # tuples containing both A & B/ total # of tuples
- Novelty – those patterns that contribute new information or increased performance to the pattern set
  - not previously known, surprising

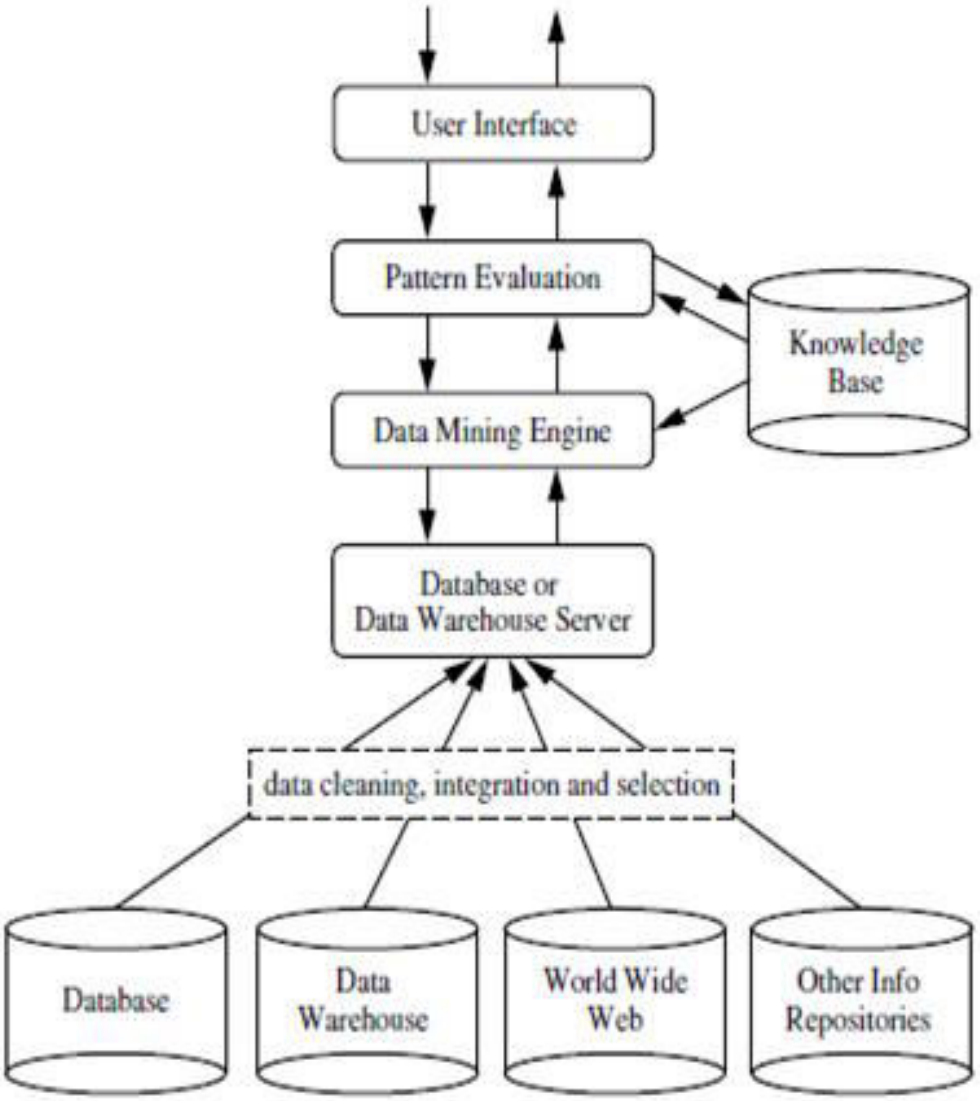
Visualization of Discovered Patterns

- Different backgrounds/usages may require different forms of representation
  - E.g., rules, tables, crosstabs, pie/bar chart etc.
- Concept hierarchy is also important
  - Discovered knowledge might be more understandable when represented at high level of abstraction
  - Interactive roll up/ drill down, pivoting, slicing and dicing provide different perspective to data
- Different kinds of knowledge require different representation: association, classification, clustering, etc.

Designing Graphical User Interfaces based on a data mining query language

- What tasks should be considered in the design (GUIs based on a data mining query language)?
  - Data collection and data mining query composition
  - Presentation of discovered patterns
  - Hierarchy specification and manipulation
  - Manipulation of data mining primitives
  - Interactive multilevel mining
  - Other miscellaneous information

Architecture of Data Mining System



Data Mining System Architectures

- Coupling data mining system with DB/DW system
  - No coupling—flat file processing, not recommended
  - Loose coupling
    - Fetching data from DB/DW
  - Semi-tight coupling—enhanced DM performance
    - Provide efficient implement a few data mining primitives in a DB/DW system, e.g., sorting, indexing, aggregation, histogram analysis, multiway join, precomputation of some stat functions
  - Tight coupling—A uniform information processing environment
    - DM is smoothly integrated into a DB/DW system, mining query is optimized based on mining query, indexing, query processing methods, etc.

Summary

- Five primitives for specification of a data mining task
  - task-relevant data
  - kind of knowledge to be mined
  - background knowledge
  - interestingness measures
  - knowledge presentation and visualization techniques to be used for displaying the discovered patterns
- Data mining system architecture
  - No coupling, loose coupling, semi-tight coupling, tight coupling